



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филiaal) _____ № 8 _____ Образовательный центр
Института № 8
Группа _____ М8О-412Б-22 _____ Направление подготовки _____ 01.03.02
Прикладная математика и информатика
Профиль _____ информатика
Квалификация _____ бакалавр

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

На тему: Использование обучение с подкреплением ИИ в позиционных играх

Автор ВКРБ _____ Андрюшин Лев Дмитриевич _____
(фамилия, имя, отчество полностью)
Руководитель _____ Аносова Наталья Павловна _____
(фамилия, имя, отчество полностью)
Консультант _____ Крылов Сергей Сереевич _____
(фамилия, имя, отчество полностью)
Консультант _____ _____
(фамилия, имя, отчество полностью)
Рецензент _____ _____
(фамилия, имя, отчество полностью)

К защите допустить

Директор Дирекции Института № 8 _____

Крылов Сергей Сереевич _____
(фамилия, имя, отчество полностью)

_____ 20 ____ г.

Москва 2026



ОД-093-СМК-РО.1-001-Ф-4, пер. 1.0
МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-409Б-22 Направление подготовки 01.03.02 Прикладная математика и информатика

Профиль Информатика

Квалификация бакалавр

УТВЕРЖДАЮ

Директор Дирекции Института № 8

(подпись)

(инициалы, фамилия)

10

февраля

2026 г.

ЗАДАНИЕ на выпускную квалификационную работу бакалавра

Обучающийся Андрюшин Лев Дмитриевич

(фамилия, имя, отчество полностью)

Руководитель Аносова Наталья Павловна

(фамилия, имя, отчество полностью)

Старший преподаватель ОЦ 8 МАИ

(ученая степень, ученое звание, должность и место работы)

1. Наименование темы Использование обучение с подкреплением ИИ в позиционных играх

2. Срок сдачи обучающимся законченной работы 11 мая 2026

3. Задание и исходные данные к работе

Разработка и реализация системы искусственного интеллекта на основе методов обучения с подкреплением для позиционной пошаговой дуэльной стратегии с последующей интеграцией в игровой программный продукт.

4. Перечень подлежащих разработке разделов и этапы выполнения работы

п/п	Наименование раздела или этапа	Трудоёмкость в % от полной трудоёмкости ВКРБ	Срок выполнения	Примечание
1	Сбор и анализ материалов	10	22.02.2026	
2	Проектирование системы	10	04.03.2026	

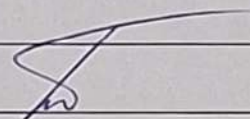
3	Разработка игровой логики	10	14.03.2026	
4	Разработка игрового ядра	20	28.03.2026	
5	Обучение и тестирование модели	35	20.04.2026	
6	Интеграция обученной модели в игровую систему	10	27.04.2026	
7	Подведение итогов ВКР	5	30.04.2026	

5. Исходные материалы и пособия

1. Osborne M. J. An Introduction to Game Theory. – New York : Oxford University Press, 2004. – 533 с.
2. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. – Cambridge, MA : MIT Press, 2018. – 552 с
3. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal Policy Optimization Algorithms. – 2017. – URL: <https://arxiv.org/abs/1707.06347> (дата обращения: 30.04.2026).

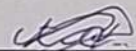
6. Дата выдачи задания 10 февраля 2026 г.

Руководитель



(подпись)

Задание принял к исполнению



(подпись)



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

О Т З Ы В
РУКОВОДИТЕЛЯ

Обучающийся Андрюшин Лев Дмитриевич

(фамилия, имя, отчество полностью)

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-409Б-22 Направление подготовки 01.03.02 Прикладная математика и информатика

Профиль Информатика

Квалификация бакалавр

Руководитель Аносова Наталья Павловна, Старший преподаватель ОЦ 8 МАИ

(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Современные методы обучения с подкреплением находят всё более широкое применение при разработке интеллектуальных игровых систем. В отличие от классических подходов, основанных на использовании заранее заданных эвристик и правил принятия решений, обучение с подкреплением позволяет формировать стратегию поведения на основе опыта взаимодействия с игровой средой. Выпускная квалификационная работа посвящена разработке системы искусственного интеллекта для оригинальной позиционной пошаговой дуэльной игры. Программная реализация выполнена с использованием языков C# и Python. Для обучения модели использована библиотека Stable Baselines3, а интеграция обученной модели в игровой продукт выполнена с применением игрового движка Unity и формата ONNX. Разработанное решение позволяет формировать игровую стратегию без использования заранее заданных эвристических правил и демонстрирует работоспособность выбранного подхода.

Основные результаты работы:

1. выполнена формализация правил разработанной игровой системы;
2. разработана архитектура системы искусственного интеллекта и программного проекта;
3. реализованы программные компоненты игровой системы и среда обучения модели;
4. выполнены обучение и тестирование модели на основе методов обучения с подкреплением;
5. разработанное решение интегрировано в игровой программный продукт.

Поставленная цель достигнута, все задачи выпускной квалификационной работы выполнены в полном объёме. Работа выполнена на высоком техническом уровне. Существенных недостатков в работе не выявлено.

Работа проверена на объем заимствования. % заимствования – 2,39

Заключение: Выпускная квалификационная работа бакалавра заслуживает оценки «отлично», а Андрюшин Л. А. – присвоения квалификации бакалавра по направлению 01.03.02 «Прикладная математика и информатика».

_____ 2026 г.

Руководитель _____

(подпись)



ОД-093-СМК-ПОД-001-Ф-7, вер. 1.0

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

**О Т З Ы В
КОНСУЛЬТАНТА**

Обучающийся Андрюшин Лев Дмитриевич

(фамилия, имя, отчество полностью)

Институт (Филиал) № 8 «Компьютерные науки и прикладная математика»

Образовательный центр Института № 8

Группа М8О-409Б-22 Направление подготовки 01.03.02 Прикладная математика и информатика

Профиль Информатика

Квалификация бакалавр

Консультант Крылов Сергей Сергеевич, к.ф.-м.н. доцент, директор института № 8

(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Выпускная квалификационная работа посвящена разработке системы искусственного интеллекта для оригинальной позиционной пошаговой дуэльной игры на основе методов обучения с подкреплением. В ходе выполнения работы были рассмотрены вопросы проектирования игровой среды, организации процесса обучения модели и интеграции обученного агента в игровой программный продукт.

В рамках консультирования были проанализированы архитектурные решения, использованные при построении программной системы, а также особенности реализации среды обучения и взаимодействия между компонентами, разработанными на языках C# и Python. Следует отметить грамотную организацию игрового окружения, корректную реализацию механизма обучения модели и успешную интеграцию полученного решения в игровой продукт на базе Unity.

Разработанная система демонстрирует возможность формирования игровой стратегии без использования заранее заданных эвристических правил и подтверждает применимость методов обучения с подкреплением для решения поставленной задачи. Полученные результаты имеют практическую значимость и могут быть использованы при дальнейшем развитии интеллектуальных игровых систем.

Замечаний по представленной работе не имеется.

Заключение: Выпускная квалификационная работа соответствует требованиям, предъявляемым к работам бакалавров по направлению 01.03.02 «Прикладная математика и информатика», и может быть рекомендована к защите. *Оценка от*

2026 г.

Консультант

(подпись)

Справка о проверке на наличие заимствований

Имя файла: ВКР_Андрюшин.docx

Автор: Андрюшин Лев Дмитриевич

Заглавие: Применение обучения с подкреплением в позиционных играх

Год публикации: 2026

Комментарий: Не указан

Коллекции: Интернет 2.0, Научные статьи 2.0, Википедия, Англоязычная Википедия, Коллекция Энциклопедий, Библиотека Либрусек, Университетская библиотека, Коллекция КФУ, ВКР Российского университета кооперации, Коллекция АПУ ФСИН, Коллекция ПГУТИ, Репозиторий открытого доступа СПб гос. ун-та, Научная электронная библиотека "КиберЛенинка", ЦНМБ Сеченова, Авторефераты ВАК, Диссертации ВАК, Диссертации РГБ, Авторефераты РГБ, Готовые рефераты, ФИПС. Изобретения, ФИПС. Полезные модели, ФИПС. Промышленные образцы, Коллекция Руконт, Библиотека им. Ушинского, Готовые рефераты (часть 2), Открытые научные источники, НЭБ, БиблиоРоссика, Правовые документы I, Правовые документы II, Правовые документы III, Собрание законодательства Российской Федерации



Результат проверки

Оценка оригинальности документа: 96%

Оригинальные фрагменты: 96,34%

Обнаруженные заимствования: 2,39%

Цитирование: 1,27%

96%

218

Работу проверил: Аносова Наталья Павловна

Дата:

18.05.2026

Подпись:

РЕФЕРАТ

Выпускная квалификационная работа состоит из 82 страниц, 7 рисунков, 10 использованных источников, 1 приложения.

ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ, ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ, ПОЗИЦИОННЫЕ ИГРЫ, ПОЗИЦИЯ, ОЦЕНКА ПОЗИЦИИ, ДЕРЕВО РЕШЕНИЙ, SELF-PLAY, DQN, PPO, НЕЙРОННЫЕ СЕТИ, ИГРОВОЕ ОКРУЖЕНИЕ, UNITY, МОДУЛЬНАЯ АРХИТЕКТУРА

Объектом исследования являются методы построения систем искусственного интеллекта для игр с полной информацией.

Объектом разработки является программная система искусственного интеллекта для позиционной пошаговой дуэльной стратегии.

Целью работы является разработка и реализация системы искусственного интеллекта на основе методов обучения с подкреплением для позиционной пошаговой дуэльной стратегии с последующей интеграцией в игровой программный продукт.

В работе использованы методы обучения с подкреплением, включая алгоритмы DQN и PPO, а также подход self-play. Реализация и обучение модели выполнены с использованием Python и библиотеки Stable Baselines3. Интеграция разработанной модели в игровой продукт осуществлена в среде Unity с использованием языка программирования C#.

В ходе работы выполнена формализация правил игры, разработана модульная архитектура системы, реализовано игровое окружение, проведено обучение модели и тестирование её эффективности. В результате разработана система искусственного интеллекта, способная принимать решения на основе текущего состояния игровой среды. Полученные результаты демонстрируют устойчивое поведение модели и подтверждают применимость методов обучения с подкреплением для решения задач данного класса.

Областью применения результатов являются игровые приложения, а также системы поддержки принятия решений в задачах с дискретным пространством состояний и полной наблюдаемостью.

Практическая значимость работы заключается в разработке программной системы, пригодной для интеграции в игровые продукты, а также в возможности повторного использования предложенной архитектуры при создании аналогичных интеллектуальных систем.

Экономическая эффективность работы не оценивалась, однако использование методов обучения с подкреплением позволяет снизить трудозатраты на разработку игровых ИИ за счёт автоматизации формирования стратегии.

В дальнейшем возможно развитие работы в направлении усложнения игровой модели, расширения пространства состояний, а также повышения эффективности алгоритмов обучения и качества принимаемых решений.

СОДЕРЖАНИЕ

РЕФЕРАТ	1
СОДЕРЖАНИЕ	4
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	6
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ.....	9
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	12
1.1 Формальная модель игры	12
1.2 Свойства корректности правил.....	13
1.3 Классификация игр в рамках теории игр.....	15
1.4 Дерево решений.....	17
1.5 Модернизация методов анализа дерева решений	18
1.5.1 Отсечение заведомо неэффективных вариантов	18
1.5.2 Ограничение глубины поиска и частичный перебор	20
1.5.3 Вывод по модернизации дерева решений	21
1.6 Обучение с подкреплением.....	21
1.7 Метод Deep Q-Network и его модификация Double DQN	23
1.8 Метод Proximal Policy Optimization (PPO)	27
2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ	32
2.1 Формализация игровой системы	32
2.2 Проверка корректности правил игровой системы.....	33
2.3 Типизация разработанной игры в рамках теории игр	35
2.4 Анализ алгоритмов принятия решений к разработанной игре.....	36
2.5 Архитектура программной системы	37
2.6 Игровое ядро и модель состояния игры	40
2.7 Взаимодействие игровой среды и алгоритм обучения.....	41
2.8 Организация процесса обучения	42
2.9 Обоснование выбора игрового движка.....	44
2.10 Игровой интерфейс и визуальное представление.....	46
2.11 Интеграция обученной модели в игровую систему	48
3 РЕАЛИЗАЦИЯ МЕТОДОВ ОБУЧЕНИЯ И РАБОТЫ СИСТЕМЫ.....	49

3.1 Работа игрового ядра в реализованной системе	50
3.2 Игровая среда и формирование взаимодействия с моделью	51
3.2.1 Представление состояния и формирование наблюдения	52
3.2.1.1 Базовые каналы	53
3.2.1.2 Сложные каналы	55
3.2.2 Пространство действий и их декодирование	57
3.2.3 Организация шага взаимодействия	58
3.2.4 Функция вознаграждения и условия завершения	60
3.3 Организация процесса обучения	62
3.4 Причины отказа от метода DQN	66
3.5 Оценка качества обученной модели	68
3.7 Применение обученной модели в игровом процессе	73
3.8 Практическая проверка качества модели	76
ЗАКЛЮЧЕНИЕ	79
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	81
ПРИЛОЖЕНИЕ А QR-код репозитория с исходным кодом	82

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе бакалавра применяются следующие термины с соответствующими определениями:

Обучение с подкреплением (reinforcement learning) – метод машинного обучения, при котором агент обучается через взаимодействие со средой и получение вознаграждений

Self-play – метод обучения, при котором агент обучается, играя против копии самого себя

Модульная архитектура – способ построения системы, при котором функциональные компоненты разделены на независимые модули

Дерево решений – это метод анализа данных и принятия решений, визуально представляющий алгоритм в виде структуры «дерева» (узлы, ветви и «листья»), который помогает классифицировать объекты или прогнозировать значения на основе набора правил «если-то»

Агент – система, принимающая решения, которая наблюдает состояние среды и выбирает действия с целью максимизации суммарного вознаграждения

Награда – числовая оценка результата действия агента, используемая для обучения

Q-learning – метод обучения с подкреплением, основанный на оценке функции ценности действий

Deep Q-Network – это алгоритм обучения с подкреплением, объединяющий Q-Learning с глубокими нейронными сетями

Temporal Difference error – ошибка временной разности между предсказанным и целевым значением функции ценности

Нейронная сеть – математическая модель, построенная по принципу организации нервных сетей (биологических нейронных сетей) – сетей нервных клеток (нейронов) живого организма

Proximal Policy Optimization – метод обучения с подкреплением, основанный на оптимизации параметризованной политики с ограничением величины обновления

Наблюдение (observation) – числовое представление текущего состояния игровой среды, используемое алгоритмом обучения

Игровой движок – программная платформа, предоставляющая инструменты для разработки и выполнения игровой логики, визуализации и взаимодействия с пользователем

Пользовательский интерфейс – совокупность средств взаимодействия пользователя с программной системой и визуального отображения состояния игры

Тензор – это многомерный числовой массив, используемый для представления входных данных и внутренних вычислений нейронной сети

Канал – это отдельная двумерная матрица фиксированного размера, являющаяся слоем многоканального тензора, в которой хранится значение одного конкретного признака для каждой позиции игрового поля

Инференс – это процесс применения обученной нейросети на практике

Коэффициент Ело — числовой показатель относительной силы игрока, рассчитываемый на основе результатов сыгранных партий

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе бакалавра применяются следующие сокращения и обозначения:

ИИ – искусственный интеллект

DQN – Deep Q-Network

PPO – Proximal Policy Optimization

TD-ошибка – Temporal Difference error

RL – Reinforcement Learning

ВВЕДЕНИЕ

В последние годы наблюдается устойчивый рост интереса к методам искусственного интеллекта, применяемым в разработке поведения компьютерного игрока в настольных позиционных играх, таких как шахматы или го. Такие игры характеризуются дискретным пространством состояний, строгими правилами перехода и необходимостью построения эффективной стратегии на основе анализа текущей конфигурации игрового поля.

Классические подходы к построению игровых стратегий, основанные на полном или частичном переборе состояний, а также на использовании эвристических правил, демонстрируют ограниченную применимость в условиях сложных игровых систем. Основной проблемой является комбинаторный рост числа возможных состояний и последовательностей действий, что делает невозможным построение полного дерева решений и его анализ за приемлемое время. Кроме того, разработка эффективных эвристик требует значительных трудозатрат и не гарантирует достижения оптимального поведения в нестандартных игровых ситуациях.

В связи с этим актуальным является применение методов обучения с подкреплением, позволяющих формировать стратегию поведения модели на основе опыта взаимодействия с игровой средой. В отличие от классических методов, данный подход не требует явного задания стратегии и способен адаптироваться к особенностям конкретной игровой модели. Он тоже не гарантирует нахождение оптимальной стратегии, однако проще в разработке и дает лучшие результаты по сравнению с классическими методами.

Объектом исследования являются методы построения систем искусственного интеллекта для игр с полной информацией. Предметом исследования являются алгоритмы обучения с подкреплением, применяемые для выработки стратегии в позиционных пошаговых играх.

Целью выпускной квалификационной работы является разработка и реализация системы искусственного интеллекта на основе методов обучения с подкреплением для позиционной пошаговой дуэльной стратегии с последующей интеграцией в игровой программный продукт.

Для достижения поставленной цели в работе решаются следующие задачи:

- проведение анализа предметной области и существующих подходов к разработке игровых интеллектуальных систем;
- формализация правил разработанной игровой системы;
- разработка архитектуры системы искусственного интеллекта и итогового проекта;
- реализация программных компонентов игровой системы и среды обучения модели;
- обучение и тестирование модели;
- интеграция разработанного решения в игровой продукт.

В работе используются методы обучения с подкреплением и методы машинного обучения. Оценка эффективности разработанной системы проводится на основе результатов тестирования модели и анализа её поведения в игровой среде. Программная реализация выполнена с использованием языка программирования C# для разработки игровой среды и Python с применением библиотеки Stable Baselines3 для обучения модели.

Научная новизна работы заключается в применении методов обучения с подкреплением для построения стратегии в разработанной позиционной пошаговой игре с оригинальными правилами формирования локальных игровых структур, а также в исследовании особенностей обучения модели в условиях self-play в данной постановке задачи.

Практическая значимость работы заключается в разработке программной системы, обеспечивающей автоматизированное формирование игровой стратегии без использования заранее заданных эвристик. Предложенная архитектура может быть использована при создании аналогичных интеллектуальных систем, а также интегрирована в игровые приложения, реализованные на базе современных игровых движков.

Таким образом, выполненная работа направлена на решение задачи построения эффективной системы искусственного интеллекта для сложной игровой среды с использованием современных методов обучения с подкреплением, что подтверждает её актуальность и практическую значимость.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1 Формальная модель игры

Игровая система рассматривается как дискретная динамическая система, поведение которой полностью определяется текущим состоянием и выбранными действиями игроков. В рамках данного исследования предполагается рассмотрение только детерминированных систем, в которых каждому допустимому действию в каждом состоянии соответствует единственный результат перехода. Формально задается кортежем:

$$G = (P, S, A, F), \quad (1)$$

где $P = \{P_1, P_2\}$ – множество игроков, S – множество состояний системы, A – множество допустимых действий, а функция перехода задается отображением:

$$F: S \times A \rightarrow S. \quad (2)$$

Здесь $F(s, a) = s'$ означает, что из состояния $s \in S$ при выполнении действия $a \in A$ систем переходит в новое состояние $s' \in S$. Такая формализация соответствует классическим моделям теории игр, в которых поведение системы описывается через последовательность переходов между состояниями [1].

Каждое состояние $s \in S$ представляет собой полное описание системы в фиксированный момент времени и может быть представлено в виде кортежа:

$$s = (B, t), \quad (3)$$

где B – конфигурация игрового пространства (например, распределение объектов на поле), а $t \in P$ – игрок, которому принадлежит текущий ход. Таким образом, состояние включает как структурную информацию о системе, так и информацию о порядке ходов.

Множество допустимых действий зависит от состояния и обозначается как $A(s) \subseteq A$. Применение действия $a \in A(s)$ приводит к переходу в новое состояние:

$$s' = F(s, a). \quad (4)$$

Последовательное применение функции перехода формирует траекторию игрового процесса:

$$s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \quad (5)$$

1.2 Свойства корректности правил

Первым свойством, непосредственно следующим из принятой детерминированной модели системы, является однозначность функции перехода [2]. Оно означает, что для любого состояния $s \in S$ и любого допустимого действия $a \in A(s)$ существует ровно одно состояние s' , такое что:

$$s' = F(s, a). \quad (6)$$

Данное свойство интерпретируется как строгая однозначность преобразования состояния: одно и то же действие в одном и том же состоянии всегда приводит к единственному результату. Это означает, что результат хода полностью определяется текущей конфигурацией системы и выбранным действием без возможных альтернативных исходов. Например, в игре «Го» размещение камня в фиксированной позиции однозначно задаёт новое состояние доски, включая возможное удаление захваченных камней.

Вторым свойством является логическая непротиворечивость правил [3]. Оно означает, что не существует состояния s , для которого некоторое условие одновременно истинно и ложно. Формально это требует, чтобы используемые предикаты $\varphi(s)$ были корректно определены:

$$\neg(\varphi(s) \wedge \neg\varphi(s)). \quad (7)$$

Здесь $\varphi(s)$ – некоторое условие, определённое на состоянии s . Это свойство гарантирует, что правила игры не содержат противоречий.

В контексте «Го» данное требование проявляется, например, в правилах захвата камней: группа камней либо имеет свободы и остаётся на доске, либо не имеет свобод и удаляется. Невозможно, чтобы одна и та же группа одновременно считалась и захваченной, и незахваченной.

Третьим важным свойством является определённость применения правил. Оно заключается в том, что результат применения действия зависит только от текущего состояния s и выбранного действия a , но не зависит от порядка вычислений или реализации алгоритма.

На примере «Го» это означает, что итоговое состояние доски после хода определяется исключительно правилами игры, а не порядком проверки условий, например, порядка обработки групп камней.

Несмотря на внешнюю схожесть, свойства детерминированности и определённости процедуры применения правил не являются тождественными. Детерминированность относится к самой математической модели и означает, что для каждого состояния и допустимого действия результат перехода однозначно определён. В то же время определённость процедуры применения правил относится к способу реализации этой модели и требует, чтобы результат не зависел от порядка выполнения вычислений или внутренних шагов алгоритма.

Различие между этими свойствами можно проиллюстрировать следующим примером. Предположим, что новое состояние определяется на основе нескольких локальных преобразований, применяемых к текущей конфигурации. Если правила сформулированы недостаточно строго, то применение этих преобразований в разном порядке может приводить к различным результатам. В этом случае формально предполагается существование единственного корректного результата, однако конкретная

реализация может его нарушать. Таким образом, детерминированность гарантирует единственность результата на уровне формального определения правил, тогда как определённости процедуры обеспечивает его корректное получение при практической реализации.

Таким образом, формализация игровой системы в виде набора основных компонентов – множества игроков, состояний, действий и функции перехода – в сочетании с требованиями детерминированности, логической непротиворечивости и определённости обеспечивает строгую математическую модель игры. Использование данных свойств позволяет гарантировать однозначность интерпретации правил и корректность моделирования игрового процесса.

1.3 Классификация игр в рамках теории игр

В рамках анализа игр в теории игр применяется их классификация по основным структурным признакам, позволяющая формально описывать различия между игровыми моделями и определять методы их дальнейшего анализа. В качестве теоретической основы используется подход, изложенный в *An Introduction to Game Theory*, где игра рассматривается как система, задаваемая множеством игроков, множеством состояний, множеством действий и правилами перехода между состояниями.

По критерию доступности информации игры подразделяются на игры с полной и неполной информацией. В играх с полной информацией все элементы текущего состояния являются наблюдаемыми для всех участников. В противоположном случае часть информации о состоянии или параметрах системы может быть скрыта, что приводит к необходимости учитывать неопределённость при принятии решений (например, карточные игры с закрытыми руками).

По критерию доступности истории различают игры с совершенной и несовершенной информацией. В играх с совершенной информацией

игрокам известна полная последовательность предыдущих ходов, тогда как в играх с несовершенной информацией часть истории или действий других участников может быть недоступна или частично скрыта, что ограничивает возможность точного восстановления процесса игры.

По наличию случайности выделяют детерминированные и стохастические игры. В детерминированных играх переходы между состояниями однозначно определяются текущим состоянием и выбранными действиями игроков. В стохастических играх в процесс дополнительно вводятся вероятностные элементы, такие как случайные события или генераторы случайных чисел, влияющие на результат перехода.

По структуре взаимодействия игроков игры делятся на последовательные и одновременные. В последовательных играх участники совершают действия по очереди, что позволяет учитывать предшествующие ходы при принятии решений. В одновременных играх выбор действий осуществляется независимо и без знания текущего выбора других игроков.

По критерию завершённости различают конечные и потенциально бесконечные игры. В конечных играх процесс гарантированно завершается за конечное число шагов либо по достижению терминального состояния. В бесконечных или неограниченных по времени моделях такое условие отсутствует, и игра может продолжаться без заранее фиксированного завершения.

По структуре симметрии игры подразделяются на симметричные и асимметричные. В симметричных играх игроки обладают одинаковыми наборами действий и одинаковыми правилами перехода, тогда как в асимметричных играх структура доступных действий или правил может различаться для разных участников, что приводит к различным стратегическим возможностям.

По типу функции выигрыша выделяются игры с нулевой и ненулевой суммой. В играх с нулевой суммой выигрыш одного игрока эквивалентен проигрышу другого, тогда как в играх с ненулевой суммой суммарный результат может не быть фиксированным, и возможны исходы, при которых выигрыши игроков не являются строго противоположными.

Таким образом, классификация игр по данным признакам позволяет формально описывать их структуру и выделять ключевые свойства, определяющие выбор методов анализа и построения стратегий.

1.4 Дерево решений

В теории игр для анализа стратегического взаимодействия используется представление в виде дерева решений. В данном формализме игра описывается как разветвлённый процесс, в котором каждая вершина соответствует возможному состоянию системы, а каждое ребро соответствует допустимому действию игрока, переводящему игру в новое состояние. Такой подход позволяет рассматривать игру как последовательность выборов, формирующих траекторию развития партии.

Дерево решений теоретически содержит все возможные последовательности ходов, начиная с начального состояния и заканчивая терминальными состояниями. Оно позволяет формально определить пространство стратегий и анализировать оптимальное поведение игроков через сравнение всех возможных вариантов развития игры.

Однако ключевым ограничением данного подхода является комбинаторный рост числа состояний. Даже при детерминированных правилах и полном отсутствии случайности число возможных позиций и последовательностей ходов увеличивается экспоненциально с глубиной дерева и числом допустимых действий. В результате полное построение дерева решений становится вычислительно невозможным уже для относительно простых стратегических игр.

Классическим примером данной проблемы является игра «Го», в которой, несмотря на простую структуру правил и полную наблюдаемость состояния, количество возможных позиций оценивается порядка 10^{170} . Это делает невозможным явный перебор всех стратегий и требует использования альтернативных методов анализа.

Таким образом, классическое дерево решений остаётся важным теоретическим инструментом описания пространства стратегий и анализа возможных траекторий развития игры. Однако его практическое применение ограничено размером пространства состояний и глубиной игры, что делает необходимым использование методов, направленных на сокращение объёма рассматриваемых состояний и приближённый анализ структуры дерева.

1.5 Модернизация методов анализа дерева решений

Для повышения применимости дерева решений в задачах со значительным пространством состояний используются методы, направленные на сокращение числа рассматриваемых вариантов и снижение вычислительной сложности поиска. Несмотря на то, что данные подходы позволяют частично адаптировать классическую модель дерева решений к практическим задачам, они не устраняют фундаментальную проблему экспоненциального роста пространства поиска, возникающую при увеличении глубины игры и числа допустимых действий.

В рамках данной работы рассматриваются два основных направления модернизации дерева решений: отсечение заведомо неэффективных вариантов и ограничение глубины анализа с использованием частичного перебора и функций оценки состояний [4].

1.5.1 Отсечение заведомо неэффективных вариантов

Метод отсечения заведомо неэффективных вариантов основан на идее сокращения пространства поиска за счёт исключения действий, которые с

высокой степенью уверенности не приводят к улучшению позиции игрока. В классическом дереве решений каждый допустимый ход порождает новую ветвь, что приводит к экспоненциальному росту числа рассматриваемых состояний. Данный подход позволяет частично контролировать этот рост, уменьшая коэффициент ветвления на каждом уровне дерева.

Практическая реализация данного подхода предполагает использование эвристических правил, позволяющих заранее оценить перспективность конкретного действия без полного развертывания последующих состояний. Такие эвристики могут учитывать локальные характеристики игрового поля, например плотность фигур, потенциальное формирование выгодных структур, либо предотвращение очевидно убыточных ситуаций. На основе этих критериев часть действий исключается из дальнейшего рассмотрения ещё до построения соответствующих поддеревьев.

Несмотря на снижение вычислительной нагрузки, данный подход имеет существенные ограничения. Основная проблема заключается в том, что оценка действия производится локально и не учитывает долгосрочные последствия в полной мере. В результате существует риск отсека стратегически оптимальных ходов, которые на текущем этапе выглядят неэффективными, но в более глубокой перспективе приводят к выигрышной позиции. Кроме того, качество работы метода напрямую зависит от корректности выбранных эвристик, что требует значительной ручной настройки и предметного анализа игровой логики.

Таким образом, отсечение снижает размер пространства поиска, но вводит дополнительную зависимость от качества априорных оценок, что ограничивает его универсальность в сложных стратегических играх.

1.5.2 Ограничение глубины поиска и частичный перебор

Второе направление модернизации связано с отказом от полного построения дерева решений и переходом к ограниченному анализу его части. В рамках данного подхода дерево разворачивается только до заданной глубины, после чего дальнейшее развитие позиции не моделируется явно. Вместо этого используется функция оценки состояния, позволяющая приближённо определить перспективность текущей позиции.

Такой подход существенно снижает вычислительную сложность анализа, поскольку исключает необходимость рассмотрения всех возможных продолжений игры до терминальных состояний. Однако это приводит к необходимости разработки функции оценки, которая должна корректно отражать стратегическую ценность позиции на основе неполной информации (что не всегда возможно).

В более развитых вариантах данного метода могут использоваться стохастические симуляции возможных продолжений игры, при которых состояние оценивается на основе усреднённых результатов разыгрываний. Это позволяет учитывать вероятностный характер развития игры и частично компенсировать ограниченную глубину анализа.

Тем не менее, данный подход сохраняет ряд принципиальных ограничений. Во-первых, точность принятия решений напрямую зависит от качества функции оценки, которая в сложных позиционных играх является нетривиальной задачей. Во-вторых, ограничение глубины приводит к тому, что стратегия формируется на основе локально ограниченного контекста, без полного учёта глобальной структуры пространства состояний. В результате поведение системы может быть нестабильным при изменении игровой ситуации или глубины анализа.

Таким образом, частичный перебор позволяет сделать анализ дерева решений вычислительно реализуемым, но не решает проблему построения

устойчивой стратегии в условиях высокой сложности игрового пространства.

1.5.3 Вывод по модернизации дерева решений

Рассмотренные методы позволяют существенно снизить вычислительную сложность анализа дерева решений за счёт сокращения пространства поиска и ограничения глубины перебора. Однако оба подхода сохраняют фундаментальные ограничения, связанные с необходимостью использования эвристических оценок и локальным характером анализа.

В частности, отсечение опирается на априорные предположения о полезности действий, а методы частичного перебора – на приближённую оценку состояний без полного анализа последствий. Это приводит к тому, что качество принимаемых решений зависит от внешне заданных функций оценки и не гарантирует оптимального поведения в сложных игровых сценариях.

В связи с этим в современных задачах анализа стратегических взаимодействий используется переход к методам, не требующим полного перечисления состояний и траекторий игры. Одним из таких подходов является обучение с подкреплением (reinforcement learning), в рамках которого модель обучается выбирать действия на основе взаимодействия со средой и получаемой обратной связью, а не на основе явного анализа всего дерева решений. Данный подход позволяет работать с задачами, в которых пространство состояний слишком велико для полного перебора, и стратегия формируется через оптимизацию поведения по наблюдаемому опыту.

1.6 Обучение с подкреплением

Обучение с подкреплением представляет собой раздел машинного обучения, в котором агент обучается взаимодействовать со средой, получая сигналы вознаграждения за совершаемые действия. В отличие от методов обучения с учителем, где используются заранее размеченные данные, в

обучении с подкреплением агент самостоятельно формирует стратегию поведения на основе опыта взаимодействия со средой. Формальная постановка задачи приведена в Reinforcement Learning: An Introduction [5].

Взаимодействие агента со средой описывается последовательностью переходов вида:

$$(s_t, a_t, r_t, s_{t+1}), \quad (8)$$

где s_t – состояние среды в моменте времени t , a_t – выбранное действие r_t – получаемая награда, s_{t+1} – следующее состояние среды.

Агентом в обучении с подкреплением называется принимающая решения система, которая на каждом шаге времени получает информацию о состоянии среды и выбирает действие в соответствии с некоторой стратегией (политикой). Основной целью агента является максимизация суммарного вознаграждения в процессе взаимодействия со средой. Средой выступает сама игра, включая текущее состояние поля и правила перехода между состояниями. Состояние описывает конфигурацию игрового поля в конкретный момент времени, а действия соответствуют возможным ходам. Награда определяется результатом действий агента и может зависеть от различных показателей.

Цель обучения формулируется как максимизация ожидаемой дисконтированной суммы наград. Данная величина определяется следующим образом:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad (9)$$

где $\gamma \in [0, 1)$ – коэффициент дисконтирования, определяющий степень значимости будущих вознаграждений относительно текущих.

Использование дисконтирования позволяет формализовать принцип предпочтения немедленных наград по сравнению с отложенными и обеспечивает возможность учёта долгосрочных последствий действий. В

рамках рассматриваемой задачи это означает, что выбор действия оценивается не только по непосредственному результату, но и по его влиянию на будущие состояния игрового поля.

Процесс обучения заключается в последовательном взаимодействии агента со средой и накоплении опыта в виде набора переходов. На основе этого опыта агент корректирует свою стратегию таким образом, чтобы увеличивать значение ожидаемой суммарной награды. Формально обучение сводится к задаче поиска стратегии π , максимизирующей величину:

$$J(\pi) = E_{\pi}[G_t], \quad (10)$$

где π задаёт правило выбора действий в зависимости от состояния среды.

Обучение с подкреплением позволяет учитывать отложенные последствия действий и формировать поведение, ориентированное на достижение долгосрочного результата. В контексте рассматриваемой игры это выражается в возможности не только реагировать на текущую ситуацию, но и формировать выгодные конфигурации для будущих ходов.

Использование данного подхода является обоснованным, поскольку он позволяет эффективно работать с большим пространством состояний, учитывать сложные зависимости между действиями и их последствиями, а также постепенно улучшать стратегию поведения на основе накопленного опыта взаимодействия со средой.

1.7 Метод Deep Q-Network и его модификация Double DQN

В задачах обучения с подкреплением одним из базовых подходов является метод Q-learning [6], в котором агент стремится оценить функцию ценности действий $Q(s, a)$, отражающую ожидаемую суммарную награду при выполнении действия a в состоянии s . Формально данная функция определяется как математическое ожидание дисконтированной суммы наград:

$$Q^{\pi}(s, a) = E[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a], \quad (11)$$

где $\gamma \in [0,1)$ – коэффициент дисконтирования.

В классическом варианте данный метод реализуется в табличной форме, где для каждой пары «состояние–действие» хранится соответствующее значение. Обновление значений производится согласно уравнению Беллмана:

$$Q(s, a) \leftarrow Q(s, a) + \eta \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right), \quad (12)$$

где η – скорость обучения.

Однако при увеличении размерности пространства состояний такой подход становится неприменимым из-за необходимости хранения и обновления огромного числа значений.

В моей игре пространство состояний определяется конфигурацией игрового поля размером 9×9 , где каждая клетка может находиться в нескольких состояниях. Это приводит к экспоненциальному росту числа возможных состояний, что делает использование табличного Q-learning невозможным на практике. Для решения данной проблемы применяется аппроксимация функции ценности с помощью нейронной сети.

Метод Deep Q-Network (DQN) [7], основан на использовании нейронной сети для приближения функции:

$$Q(s, a; \theta), \quad (13)$$

где θ – параметры сети. В данном подходе нейронная сеть принимает на вход представление состояния среды и возвращает оценки ценности для всех возможных действий.

Обучение сети осуществляется путём минимизации квадратичной функции потерь:

$$L(\theta) = E_{(s,a,r,s')} \left[(y - Q(s, a; \theta))^2 \right], \quad (14)$$

где целевое значение y вычисляется как:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-), \quad (15)$$

а θ^- – параметры целевой сети (target network) фиксируемые на протяжении некоторого числа итераций обучения.

Это позволяет агенту обобщать опыт и эффективно работать с большим пространством состояний.

Однако классический DQN имеет существенный недостаток, связанный с переоценкой значений функции полезности. Данная проблема возникает из-за того, что при обновлении оценок используется одно и то же приближение как для выбора наилучшего действия, так и для оценки его качества. В результате значения Q-функции могут систематически завышаться, что негативно влияет на стабильность и качество обучения.

Для устранения данной проблемы была предложена модификация алгоритма – Double Deep Q-Network (Double DQN) [8].

Основная идея данного метода заключается в разделении процесса выбора действия и его оценки. В Double DQN используются две нейронные сети: одна применяется для выбора наилучшего действия в текущем состоянии, а другая – для оценки его ценности. Целевое значение при этом вычисляется следующим образом:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta), \theta^-). \quad (16)$$

Такое разделение позволяет снизить эффект переоценки, поскольку выбор действия и вычисление его полезности выполняются разными моделями. В результате обучение становится более стабильным, а получаемая стратегия – более точной. Это особенно важно в задачах с

большим пространством состояний и сложной зависимостью между действиями и результатами, к которым относится моя игра.

Дополнительно для повышения стабильности и эффективности обучения в рамках DQN используются специальные механизмы, позволяющие улучшить процесс обновления нейронной сети. Одним из таких механизмов является буфер воспроизведения опыта (experience replay).

Идея experience replay заключается в том, что агент сохраняет свои взаимодействия со средой в виде набора переходов «состояние – действие – награда – следующее состояние» (s, a, r, s') . В дальнейшем обучение производится не на последних полученных данных, а на случайной выборке из этого буфера:

$$(s_i, a_i, r_i, s'_i) \sim D. \quad (17)$$

Это позволяет устранить сильную корреляцию между последовательными состояниями, которая возникает при непосредственном обучении на текущем опыте, и тем самым повышает устойчивость обучения. Кроме того, повторное использование прошлых переходов увеличивает эффективность обучения, так как каждый полученный опыт может быть использован многократно.

В более продвинутом варианте используется приоритизированный буфер воспроизведения [9], при котором различные переходы имеют разную вероятность быть выбранными для обучения. Приоритет, как правило, определяется величиной ошибки предсказания (temporal difference error):

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a). \quad (18)$$

Чем больше текущая оценка отличается от ожидаемой, тем более «важным» считается данный переход. Это позволяет агенту уделять больше внимания

наиболее информативным ситуациям, что ускоряет процесс обучения и способствует более быстрому улучшению стратегии.

Ещё одним важным элементом алгоритма является использование целевой сети (target network). В классическом DQN обновление параметров происходит на основе текущих оценок нейронной сети, что может приводить к нестабильности, поскольку целевые значения изменяются одновременно с обучаемой моделью. Для решения этой проблемы вводится отдельная сеть, параметры которой фиксируются на протяжении некоторого числа шагов и периодически обновляются на основе основной сети:

$$\theta^- \leftarrow \theta. \quad (19)$$

Использование целевой сети позволяет стабилизировать процесс обучения, так как значения, к которым стремится модель, остаются относительно неизменными в течение определённого времени. Это уменьшает вероятность расходимости и делает процесс оптимизации более устойчивым. В сочетании с буфером воспроизведения опыта данный механизм является стандартным элементом современных реализаций DQN.

1.8 Метод Proximal Policy Optimization (PPO)

Метод Proximal Policy Optimization (PPO) [10] относится к классу методов обучения с подкреплением, основанных на прямой оптимизации параметризованной стратегии поведения агента. В отличие от подходов, в которых обучение сводится к оценке функции полезности действий и последующему выбору максимизирующего действия, здесь стратегия задаётся явно в виде стохастической политики $\pi_\theta(a|s)$, определяющей распределение вероятностей выбора действий в зависимости от состояния среды. Такой способ формализации позволяет напрямую оптимизировать поведение агента, минуя промежуточное восстановление функции ценности действий.

Цель обучения заключается в максимизации ожидаемой суммарной дисконтированной награды, получаемой в процессе взаимодействия со средой. Данная величина задаётся выражением:

$$J(\theta) = E_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad (20)$$

где τ обозначает траекторию взаимодействия агента со средой, а коэффициент $\gamma \in [0, 1)$ определяет степень убывания значимости будущих вознаграждений. Такая постановка отражает необходимость учитывать долгосрочные последствия принимаемых решений, поскольку влияние одного действия может проявляться на последующих этапах эпизода.

Оптимизация стратегии осуществляется с использованием градиентных методов, в которых параметры политики изменяются в направлении, увеличивающем ожидаемую награду. В общем виде градиент целевой функции выражается как:

$$\nabla_\theta J(\theta) = E_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t | s_t) A_t], \quad (21)$$

где A_t представляет собой функцию преимущества, характеризующую относительную ценность конкретного действия по сравнению со средним поведением стратегии в данном состоянии. Использование данной величины позволяет нормировать вклад отдельных действий в обновление политики и снижать дисперсию градиентной оценки, что критически важно при обучении на стохастических траекториях.

Функция преимущества не наблюдается напрямую и должна быть оценена на основе взаимодействия со средой. Для этого используется временная разность, отражающая ошибку предсказания функции ценности после одного шага перехода. Базовое выражение временной разности задаётся следующим образом:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (22)$$

где функция $V(s)$ оценивает ожидаемую суммарную награду при нахождении в состоянии s . Однако использование только одношаговой ошибки приводит к высокой дисперсии оценки, поэтому в PPO применяется обобщённая оценка преимущества, учитывающая несколько последующих временных шагов:

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad (23)$$

где параметр $\lambda \in [0,1]$ регулирует компромисс между смещением и дисперсией оценки. При малых значениях λ оценка становится более локальной, при больших – учитывает более длинные зависимости во времени.

Особенностью PPO является введение ограничения на изменение стратегии между последовательными итерациями обучения. Это связано с тем, что прямое применение policy gradient может приводить к чрезмерным обновлениям параметров, что вызывает деградацию ранее выученной политики. Для контроля величины обновления используется отношение вероятностей выбора действия новой и старой стратегией:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}. \quad (24)$$

Данное отношение позволяет количественно оценить степень изменения политики относительно предыдущего состояния параметров.

Для стабилизации обучения вводится ограниченная целевая функция, в которой используется операция усечения значений отношения политик. Она определяется как:

$$L^{CLIP}(\theta) = E[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (25)$$

где параметр ϵ задаёт допустимый диапазон изменения политики. Такая форма функции исключает влияние слишком больших обновлений

градиента, если они приводят к выходу за пределы заданного интервала, сохраняя при этом полезные направления оптимизации.

Параллельно с оптимизацией стратегии в PPO производится обучение функции ценности состояния, используемой для вычисления преимущества. Ошибка аппроксимации функции ценности задаётся квадратичной функцией:

$$L^{VF}(\theta) = E \left[(V_\theta(s_t) - V_t^{\text{target}})^2 \right], \quad (26)$$

где целевое значение функции ценности формируется на основе оценённого преимущества:

$$V_t^{\text{target}} = A_t + V(s_t). \quad (27)$$

Данная компонента обеспечивает согласованность между оценкой состояния и фактическим возвратом, наблюдаемым в процессе взаимодействия со средой.

Итоговая функция оптимизации PPO объединяет несколько компонент, каждая из которых выполняет отдельную роль в процессе обучения. Помимо основного критерия оптимизации и функции ценности, используется энтропийная регуляризация, направленная на поддержание стохастичности политики и предотвращение преждевременной сходимости к детерминированному поведению:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{VF}(\theta) + c_2 E[\mathcal{H}(\pi_\theta(\cdot | s_t))], \quad (28)$$

где энтропия стратегии определяется как:

$$\mathcal{H}(\pi_\theta(\cdot | s)) = - \sum_a \pi_\theta(a|s) \log \pi_\theta(a|s). \quad (29)$$

Таким образом, процесс обучения РРО представляет собой последовательную процедуру взаимодействия со средой, в ходе которой стратегия итеративно уточняется на основе ограниченных обновлений параметров. Введение ограничения на изменение политики позволяет поддерживать баланс между улучшением стратегии и сохранением ранее достигнутого качества поведения, что обеспечивает устойчивость обучения в задачах с высокой размерностью пространства состояний.

2 ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Формализация игровой системы

Игровая система задаётся состоянием игрового поля и правилами его изменения в ходе выполнения действий игроками. Состояние игры описывается кортежем (B, t) , где B – матрица размером 9 на 9, задающая конфигурацию поля, а $t \in \{P_1, P_2\}$ – игрок, совершающий текущий ход. Клетка $B[i][j]$ может находиться в одном из следующих состояний: пустая ($\emptyset$), занята треугольником игрока $P_k(T_k)$ или квадратом игрока $P_k(S_k)$, где $k \in \{1, 2\}$.

На своем ходе игрок выбирает пустую клетку (i, j) и размещает фигуру типа $type \in \{\text{треугольник, квадрат}\}$. После выполнения действия производится обновление состояния: новая фигура записывается в выбранную клетку, меняется текущий игрок, после чего выполняется проверка области 3 на 3 с центром в только что поставленной клетке, а так же всех областей 3 на 3, где центром является клетка, соседняя с той клеткой (и по вертикали, и по горизонтали, и по диагонали), где только что была размещен фигура.

Для формализации понятия «область» введем окрестность каждой клетки $c = (i, j)$:

$$N(c) = (x, y) \parallel x - i \leq 1, |y - j| \leq 1, (x, y) \neq (i, j). \quad (30)$$

Если соседняя клетка выходит за границы поля, она считается нейтральной и учитывается как занятой элемент. Определим функцию занятости $occ(x, y)$ следующий образом:

$$occ(x, y) = \begin{cases} 1, & \text{если клетка занята или является границей} \\ 0, & \text{если клетка пуста} \end{cases}. \quad (31)$$

Область с центром в клетке c считается закрытой, если выполняются два условия. Первое условие – полная занятость окрестности:

$$\sum_{(x,y) \in N(c)} occ(x,y) = |N(c)|. \quad (32)$$

Второе условие – определенное количество фигур нужного типа вокург центра: для треугольника (Т) центр должен иметь ровно три треугольника в окрестности, для квадрата (S) не менее четырех квадратов. При этом кому принадлежат фигуры в окрестности значения не имеет. Это исключает любые логические конфликты в определении закрытой области.

Пусть k_1 и k_2 – количество областей, закрытых игроками P_1 и P_2 на данном ходе. Если $k_1 > k_2$, победителем становится P_1 ; если $k_2 > k_1$, победителем становится P_2 . В случае если $k_1 = k_2$ игра продолжается. Максимальное количество ходов ограничено 80, если после 80 хода победителя не выявлено, фиксируется ничья. Так как поле 9 на 9, что при умножении дает 81, а максимум ходов 80, игрокам всегда будет куда поставить фигуру.

2.2 Проверка корректности правил игровой системы

Корректность игровой системы рассматривается как выполнение ряда формальных свойств на уровне конкретной реализации правил перехода состояния и процедуры оценки ходов.

Первое свойство – детерминированность перехода состояния. Проявляется в том, что результат любого допустимого действия однозначно определяется текущей конфигурацией поля и выбранным ходом. В рамках разработанной системы действие игрока задаётся строго фиксированными параметрами: координатами клетки и типом фигуры. После этого выполняется последовательная процедура обновления состояния: запись фигуры в поле, смена активного игрока и запуск проверки локальных областей. Поскольку каждый этап преобразования состояния не содержит случайных или альтернативных ветвлений, одно и то же входное состояние и действие всегда приводят к единственному результату.

Второе свойство – логическая непротиворечивость правил игры. Заключается в отсутствии конфликтующих условий внутри формализованной системы правил, при которых одно и то же состояние игрового поля могло бы допускать несколько взаимоисключающих интерпретаций результата применения правил. В рамках разработанной системы это обеспечивается тем, что все правила преобразования и анализа состояния определены через фиксированные, однозначные критерии, применяемые к единому текущему состоянию без альтернативных сценариев вычисления. Таким образом, каждая игровая конфигурация однозначно соотносится с результатом применения правил, что исключает возможность логических противоречий внутри механизма принятия решения.

Третье свойство – однозначность процедуры проверки для любого действия. Обеспечивается тем, что анализ состояния выполняется в строго фиксированной последовательности после завершения хода и изменения конфигурации игрового поля. Проверка осуществляется над уже сформированным состоянием без его модификации в процессе вычислений, а множество проверяемых структур однозначно определяется положением совершённого хода. Это исключает зависимость результата проверки от порядка обработки элементов и гарантирует воспроизводимость результата при одинаковом входном состоянии.

Таким образом, совокупность рассмотренных свойств обеспечивает формальную корректность игровой системы на уровне правил перехода и процедуры анализа состояния: каждое действие приводит к однозначному результату, логика правил не допускает альтернативных интерпретаций, а механизм проверки сохраняет стабильность и воспроизводимость при фиксированном состоянии игрового поля.

2.3 Типизация разработанной игры в рамках теории игр

В рамках анализа разработанной игры целесообразно выполнить её классификацию по основным признакам, используемым в теории игр. Подобная классификация позволяет формально описать структуру взаимодействия игроков и определить класс моделей, к которому относится рассматриваемая система. В качестве теоретической основы используется стандартная классификация игр, в рамках которой учитываются свойства информации, наличие случайности, структура ходов и критерии завершения.

Разработанная игра относится к классу игр с полной информацией, поскольку состояние игрового поля полностью наблюдаемо для обоих участников на каждом шаге. Все элементы конфигурации доступны игрокам без ограничений, что исключает наличие скрытых параметров состояния.

Кроме того, игра обладает свойством совершенной информации, так как каждый игрок при принятии решения располагает полной историей предыдущих ходов. Последовательность действий является наблюдаемой и однозначно восстанавливаемой, что исключает неопределённость, связанную с частично скрытой динамикой игры.

С точки зрения наличия случайности игра является детерминированной. Об этом уже было сказано ранее.

По структуре взаимодействия игра относится к классу последовательных игр, в которых действия игроков выполняются поочерёдно, а выбор стратегии на каждом ходу осуществляется с учётом уже совершённых действий оппонента.

Отдельной характеристикой является ограниченность процесса игры. Введение верхнего предела на количество ходов обеспечивает завершение игрового процесса за конечное число ходов, что исключает возможность бесконечного продолжения партии.

С точки зрения симметрии игра является симметричной, поскольку оба игрока обладают идентичными наборами действий и находятся в равных стартовых условиях, за исключением порядка совершения хода, что является стандартным элементом пошаговых моделей.

По критерию структуры выигрыша игра не может быть строго отнесена к играм с нулевой суммой, поскольку её результат определяется совокупностью локальных исходов, а не прямым перераспределением единственного числового выигрыша между игроками. При этом возможны ситуации равного результата, а также ничейный исход.

Таким образом, разработанная система относится к классу конечных, детерминированных, симметричных, последовательных игр с полной и совершенной информацией, не сводящихся к строго нулевой сумме.

2.4 Анализ алгоритмов принятия решений к разработанной игре

Применение полного дерева решений к разработанной игре сталкивается с вычислительными ограничениями, связанными с размером пространства состояний и характером переходов между ними.

Игровое поле имеет размер 9×9 , что задаёт 81 клетку, каждая из которых может находиться в одном из нескольких дискретных состояний. Это приводит к оценке числа возможных конфигураций порядка:

$$5^{81} \approx 4.135 \times 10^{56}$$

Даже если учитывать различные факторы, уменьшающие это число, например то, что многие позиции на поле не могут возникнуть просто потому, что победа кого-либо из игроков возникла бы раньше, чем возникли бы эти состояния или учитывать зеркальность многих позиций, это число уменьшается незначительно. Количество различных последовательностей действия остается астрономическим. На каждом ходе количество допустимых действий определяется формулой:

где i – номер уже прошедшего количества ходов. Это приводит к быстрому росту числа возможных продолжений уже на первых уровнях анализа, что делает построение полного дерева решений невозможным.

В связи с этим в условиях столь большого пространства состояний, как было показано в теоретическом разделе, для построения стратегии применяется подход на основе обучения с подкреплением.

2.5 Архитектура программной системы

Программная система разработана на основе модульной архитектуры, разделяющей основные функциональные компоненты на независимые слои: игровую логику, визуализацию и модуль обучения искусственного интеллекта. Такой подход обеспечивает изоляцию компонентов, упрощает их тестирование и позволяет использовать их независимо в различных режимах работы системы.

Основным компонентом системы является игровое ядро, реализующее правила разработанной позиционной игры. Оно отвечает за хранение и обновление состояния игрового поля, обработку ходов, проверку их допустимости, а также определение условий завершения партии. Игровое ядро является единственным источником истинного состояния игры и не зависит от визуализации или алгоритмов машинного обучения, что обеспечивает единообразное применение правил во всех режимах системы.

Визуализация игрового процесса реализована с использованием Unity. Данный компонент обеспечивает отображение состояния игры и взаимодействие пользователя с системой в режиме ручного управления. Unity получает текущее состояние игрового поля от игрового ядра, отображает его в графическом виде и передаёт пользовательские действия

для дальнейшей обработки. При этом визуализация не содержит игровой логики и не влияет на правила выполнения ходов.

Для взаимодействия между игровым ядром и алгоритмом обучения реализован отдельный программный компонент среды, выполняющий функцию интерфейса между системой и агентом (агентом в рассматриваемой задаче является игровой алгоритм, принимающий решения о размещении фигур на поле). Он формирует тензорное представление состояния игрового поля и обеспечивает выполнение действий агента через стандартный цикл взаимодействия “состояние–действие–новое состояние”. Данный компонент определяет способ представления игры с точки зрения алгоритма обучения и обеспечивает согласованность взаимодействия между всеми частями системы.

Алгоритм обучения искусственного интеллекта реализован на языке Python и представляет собой отдельный программный модуль, отвечающий за формирование стратегии поведения агента. В процессе работы алгоритм принимает состояние игровой среды, выбирает действие, получает результат взаимодействия и использует полученный опыт для обновления модели поведения. С точки зрения архитектуры данный компонент рассматривается как автономный модуль принятия решений.

В процессе разработки выделяются два основных этапа работы системы: этап обучения и этап использования обученной модели. На этапе обучения игровое ядро и модуль среды взаимодействуют с алгоритмом обучения, обеспечивая многократное взаимодействие агента с игровой средой. В результате этого процесса формируется обученная модель поведения.

После завершения обучения модель используется интегрируется в игровую систему. В этом режиме игровое ядро передаёт текущее состояние игры модели агента, которая возвращает выбранное действие. Полученные

действия применяются в игровом ядре, а визуализация состояния осуществляется через Unity, обеспечивая пользовательское взаимодействие с системой. На рисунке 1, представлена планируемая архитектура проекта:



Рисунок 1 – Архитектура проекта

Теперь предлагаю подробно рассмотреть все элементы архитектуры по отдельности, а так же рассмотреть отдельно рассмотреть процесс обучения модели. Начнем с игрового ядро, так как оно является центральной частью всего проекта.

2.6 Игровое ядро и модель состояния игры

Игровое ядро реализует структурированное представление состояния игровой системы, обеспечивающее хранение текущей конфигурации игрового поля и поддержку операций его изменения в соответствии с заданными правилами.

Состояние игрового процесса задаётся конфигурацией игрового поля и набором служебных параметров. Игровое поле представлено двумерной сеткой фиксированного размера, каждая ячейка которой содержит информацию о своём текущем состоянии. Внутренняя область поля соответствует игровой зоне, тогда как внешние элементы используются для корректной обработки граничных ситуаций при анализе окрестностей.

Содержимое ячеек определяется наличием или отсутствием игровой фигуры. Фигура характеризуется типом и принадлежностью игроку. Тип задаёт условия участия в формировании игровых конфигураций, а принадлежность используется при учёте результата. Пустые ячейки и граничные элементы рассматриваются как отдельные состояния, учитываемые при проверке условий игры.

Дополнительно в состоянии фиксируются текущий игрок, счёт и количество выполненных ходов. Эти параметры используются для управления порядком ходов, определения результата партии и ограничения её продолжительности.

Изменение состояния осуществляется путём размещения фигуры в допустимой позиции игрового поля. В результате выполнения хода обновляется конфигурация поля и происходит переход к следующему игроку. После каждого хода выполняется проверка областей, связанных с позицией установленной фигуры.

Проверка сводится к рассмотрению фиксированного набора областей размером 3×3 , центры которых совпадают с клеткой размещения фигуры и

соседними с ней клетками. Для каждой такой области последовательно проверяются условия её закрытия, определяемые типом центральной фигуры и конфигурацией окружающих клеток.

При выполнении условий закрытия соответствующей области производится начисление очков игроку, которому принадлежит центральная фигура. Если условия не выполняются, состояние счёта не изменяется. Таким образом, после каждого хода выполняется локальная проверка условий закрытия областей и, при необходимости, обновляется счёт без пересчёта всего игрового поля.

2.7 Взаимодействие игровой среды и алгоритм обучения

В рамках системы реализован интерфейс взаимодействия между игровым ядром и алгоритмом обучения, предназначенный для представления игровой логики в форме, пригодной для методов обучения с подкреплением. Данный слой формирует абстракцию среды, обеспечивающую единый протокол обмена информацией между игровой системой и внешним алгоритмом принятия решений.

Игровая среда в рамках данной архитектуры представляет собой промежуточный компонент между внутренней логикой игры и алгоритмом обучения. Игровое ядро отвечает за реализацию правил и изменение состояния игрового поля, тогда как среда обеспечивает представление этого состояния в стандартизированном виде, а также преобразование выбранных действий в изменения состояния системы.

Ключевой функцией среды является формирование наблюдения, которое представляет собой «многоканальное» (вектор состояний, обычно большой размерности) описание текущего состояния игрового поля. Такое представление включает как базовую информацию о размещении игровых элементов, так и дополнительные признаки, отражающие локальные структурные характеристики состояния. Это позволяет алгоритму обучения

работать не с абстрактной структурой игрового ядра, а с унифицированным числовым представлением состояния.

Дополнительно среда обеспечивает управление пространством допустимых действий. Для каждого состояния формируется множество корректных ходов, зависящее от текущей конфигурации игрового поля. Это множество используется для исключения недопустимых действий и ограничения выбора на уровне взаимодействия со средой, что повышает стабильность процесса принятия решений.

Отдельным элементом взаимодействия является механизм перехода между состояниями. Выполнение действия приводит к изменению состояния игрового поля в соответствии с правилами игры, после чего формируется новое наблюдение и вычисляется результат перехода, включающий информацию о достижении терминального состояния и численную оценку результата.

Таким образом, среда выполняет роль формального интерфейса, объединяющего игровую логику и алгоритм обучения в единую систему взаимодействия. Она обеспечивает строгую структуризацию процесса обмена информацией, включающего текущее состояние, допустимые действия и результат перехода, что позволяет использовать игровой процесс в рамках методов обучения с подкреплением без изменения внутренней логики игрового ядра.

2.8 Организация процесса обучения

В рамках разработанной системы формирование стратегии не задаётся в явном виде, а определяется в результате процесса обучения, основанного на взаимодействии с игровой средой. Обучение рассматривается как составная часть системы, обеспечивающая переход от формального описания правил игры к построению эффективного поведения в пространстве состояний.

Процесс обучения организован как итеративная процедура взаимодействия с игровой средой. На каждом шаге используется текущее представление состояния, на основе которого выбирается действие, после чего выполняется переход к новому состоянию и формируется оценка результата выполненного действия. Последовательность таких шагов образует поток опыта, используемый для последующего улучшения стратегии.

Текущее состояние игровой системы преобразуется в наблюдение, доступное в рамках процедуры обучения. На его основе выбирается действие из множества допустимых, определяемого текущим состоянием. Применение действия приводит к переходу в новое состояние, сопровождаемому численной оценкой результата. Указанные элементы формируют базовую единицу данных, на основе которой осуществляется обновление стратегии.

Существенным элементом является учёт ограничений на множество допустимых действий. В каждом состоянии доступен лишь поднабор всех возможных действий, определяемый правилами игры. Исключение недопустимых действий на уровне взаимодействия со средой позволяет сократить пространство выбора и обеспечивает корректность формируемой стратегии, предотвращая рассмотрение заведомо невозможных ходов.

Организация обучения ориентирована на работу в условиях большого пространства состояний, при котором полный перебор всех возможных позиций и траекторий развития игры является вычислительно недостижимым. В связи с этим процесс обучения должен обеспечивать формирование обобщённых зависимостей между состояниями и действиями, позволяющих принимать решения в ранее не встречавшихся ситуациях. Существенным требованием является также способность к итеративному улучшению стратегии по мере накопления опыта взаимодействия.

Дополнительно в системе применяется подход самосоревнования (self-play), при котором стратегия обучается путем последовательных игр против собственных предыдущих версий. Такой механизм позволяет постепенно повышать сложность оппонента в процессе обучения и обеспечивает адаптацию стратегии без необходимости заранее фиксировать набор внешних противников. Это способствует более устойчивому улучшению качества принимаемых решений.

Таким образом, обучение в разработанной системе представляет собой процесс последовательного накопления и использования опыта взаимодействия с игровой средой, направленный на формирование эффективной стратегии поведения в условиях ограниченности вычислительных ресурсов и высокой комбинаторной сложности задачи.

2.9 Обоснование выбора игрового движка

В качестве платформы для реализации пользовательской части системы выбран игровой движок Unity. Выбор обусловлен необходимостью сочетания средств визуализации, удобной организации игровой логики и возможности интеграции с внешними вычислительными модулями, реализующими алгоритмы обучения.

При выборе технологической основы рассматривались несколько альтернативных подходов: Unity, Unreal Engine, Godot, а также реализация пользовательской части на базе низкоуровневых графических библиотек (OpenGL/DirectX) либо в виде десктопного приложения с минимальной графикой (например, на основе стандартных средств .NET).

Unreal Engine представляет собой мощный игровой движок, ориентированный на высококачественную 3D-графику и крупномасштабные проекты. Его основным преимуществом является высокая производительность и развитая система визуализации. Однако в контексте данной работы его использование приводит к избыточной

сложности: значительная часть функциональности ориентирована на реалистичную графику и крупные игровые сцены, тогда как основная задача проекта связана с моделированием и обучением стратегии, а не с графической составляющей. Кроме того, интеграция внешних алгоритмов обучения требует дополнительных усилий из-за более тяжёлой архитектуры движка.

Godot Engine является более лёгкой и гибкой альтернативой с открытым исходным кодом. Он предоставляет удобную среду для разработки 2D/3D приложений и отличается низким порогом входа. Однако на момент реализации проекта экосистема Godot менее развита в части интеграции с внешними машинно-обучающими фреймворками и промышленными инструментами, а также имеет ограниченную поддержку готовых решений для сложных систем взаимодействия между симуляцией и внешними вычислительными процессами.

Низкоуровневые графические библиотеки (OpenGL/DirectX) обеспечивают максимальную гибкость и контроль над рендерингом, однако требуют самостоятельной реализации всех компонентов прикладной системы: управления сценой, обработки ввода, архитектуры объектов, логики обновления состояния и визуализации. Это приводит к существенному увеличению объёма инфраструктурного кода, не связанного напрямую с исследуемой задачей обучения, и значительно замедляет разработку.

Консольная реализация на базе стандартных средств (.NET Console) использовалась на раннем этапе разработки для проверки корректности работы игровой логики и отладки основных механик. Данный формат позволял быстро тестировать функциональность системы, контролировать выполнение правил игры и оперативно вносить изменения в алгоритмы без необходимости реализации дополнительной инфраструктуры отображения. По мере развития проекта стало очевидно, что текстовое представление

состояния игры ограничивает удобство взаимодействия с системой, так как требует ручного анализа большого объёма информации и не обеспечивает наглядного отображения изменений состояния. В связи с этим дальнейшая разработка была ориентирована на использование графического представления, обеспечивающего более удобное восприятие игрового процесса и упрощение анализа корректности поведения системы.

На фоне перечисленных альтернатив Unity обладает рядом существенных преимуществ. Он предоставляет готовую компонентную архитектуру, включающую систему объектов, сцен и обновляемых компонентов, что позволяет логически разделять игровую логику, визуализацию и управляющие модули. Дополнительно Unity обеспечивает удобную интеграцию с внешними программными компонентами через стандартные интерфейсы, что критично для подключения алгоритмов обучения без модификации игровой логики.

Таким образом, Unity выбран как наиболее сбалансированное решение, обеспечивающее оптимальное соотношение между уровнем абстракции, трудоёмкостью разработки и возможностью расширения системы. Он позволяет сосредоточиться на реализации и исследовании алгоритмов принятия решений, минимизируя затраты на реализацию вспомогательной инфраструктуры.

2.10 Игровой интерфейс и визуальное представление

Пользовательский интерфейс системы обеспечивает визуальное представление игрового процесса и организацию взаимодействия пользователя с игровой логикой. Основным элементом отображения является игровое поле, на котором полностью отражается текущее состояние системы. Состояние игры задаётся расположением игровых объектов на поле и не требует дополнительных визуальных индикаторов для интерпретации ситуации.

Структура интерфейса реализована через набор последовательных сцен, соответствующих основным этапам взаимодействия с системой. Первая сцена представляет главное меню, в котором осуществляется выбор режима игры. Предусмотрены режимы игры между двумя пользователями за одним устройством, а также режим игры против компьютерного оппонента.

В режиме игры против компьютерного оппонента дополнительно предусмотрена сцена настройки параметров партии, в рамках которой задаются основные параметры взаимодействия, включая выбор стороны игрока и конфигурации используемой модели. Таким образом, данный режим отличается наличием предварительного этапа настройки перед началом игровой сессии.

Взаимодействие с игровой сценой осуществляется через выбор типа фигуры и указание целевой позиции на игровом поле. Управление реализовано в виде прямого выбора клетки, что обеспечивает однозначное соответствие между пользовательским вводом и изменением состояния игровой системы.

Визуальное представление построено таким образом, чтобы минимизировать необходимость дополнительной интерпретации состояния. Игровое поле является единственным источником информации о текущей ситуации в системе, что обеспечивает простую и однозначную структуру отображения.

Разделение интерфейсной части и игровой логики позволяет изолировать визуальное представление от правил игры и внутренней модели состояния. Это обеспечивает модульность системы и упрощает её дальнейшее развитие и расширение

2.11 Интеграция обученной модели в игровую систему

Интеграция обученной модели в итоговую систему заключается во включении уже обученного алгоритма в структуру игрового приложения и обеспечении его взаимодействия с игровым процессом и пользовательским интерфейсом.

В рамках реализации модель подключается как отдельный компонент, отвечающий только за вычисление ходов в режиме игры против компьютерного оппонента. Она не изменяет игровую логику и не участвует в её выполнении, а используется как внешний модуль принятия решений.

Для взаимодействия с игровой системой реализован отдельный программный слой, который выполняет загрузку модели и организацию её вызова в момент хода компьютера. Этот слой получает текущее состояние игры из игрового ядра, передаёт его в модель и возвращает выбранное действие обратно в систему.

Помимо этого, в игровую систему были внесены необходимые изменения для поддержки режима игры против модели. В частности, добавлена возможность выбора режима игры и настройки параметров компьютерного оппонента, включая выбор используемой версии модели и стороны, за которую играет пользователь.

Также обеспечено взаимодействие между логикой игры и пользовательским интерфейсом, позволяющее отображать ход, выполненный моделью, на игровом поле в рамках стандартного игрового цикла.

Таким образом, интеграция модели представляет собой включение готового алгоритма принятия решений в структуру приложения без изменения правил игры, с организацией обмена данными между игровой логикой, моделью и интерфейсом пользователя.

3 РЕАЛИЗАЦИЯ МЕТОДОВ ОБУЧЕНИЯ И РАБОТЫ СИСТЕМЫ

В предыдущей главе была рассмотрена структура разрабатываемой системы, включая описание игровых правил, архитектуры и способов организации взаимодействия между её компонентами. Особое внимание было уделено формализации игровой среды и определению принципов, на которых строится процесс обучения.

В настоящей главе рассматривается практическая реализация указанных решений. Описывается работа ключевых компонентов системы, включая игровое ядро, игровую среду и процесс обучения, а также анализируются особенности применения обученной модели в рамках игрового процесса. Основной акцент делается на том, как принятые на этапе проектирования решения реализуются на практике и обеспечивают функционирование системы в целом. Ссылка на репозиторий с исходным кодом приведена в приложении А. На рисунке 2 представлена диаграмма классов, отражающая структуру основных компонентов программной реализации и связи между ними:

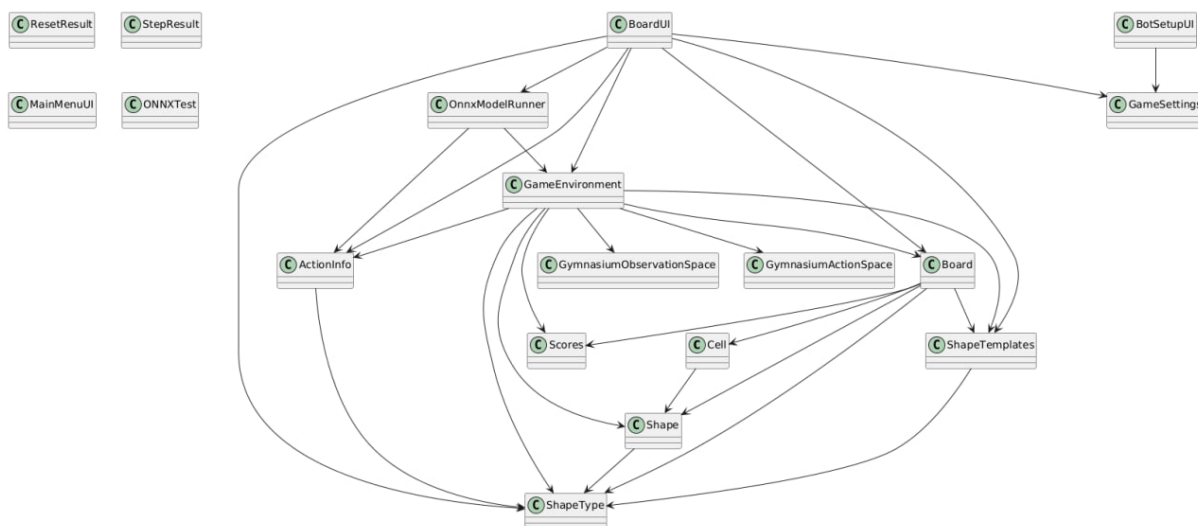


Рисунок 2 – Диаграмма классов

3.1 Работа игрового ядра в реализованной системе

Игровое ядро реализовано в виде набора классов, основными из которых являются Board, Cell, Shape и ShapeType. Класс Board отвечает за хранение и изменение состояния игры и содержит двумерный массив Cells, представляющий игровое поле размером Size × Size. Каждая ячейка (Cell) хранит текущее состояние через поле CurrentShape.

Типы фигур задаются через класс ShapeType, а конкретные варианты фигур определены в ShapeTemplates (например, Cube, TrianglePrism, Border). Это позволяет в логике игрового ядра оперировать не конкретными объектами, а их типами через параметр BaseType.

Инициализация игрового поля выполняется в конструкторе Board, где создаются все ячейки и формируется граничная область с использованием ShapeTemplates.Border. Границы заполняются заранее через вызовы метода MakeMove, что упрощает последующую обработку соседних клеток и исключает необходимость дополнительных проверок на выход за пределы поля.

Основная логика хода реализована в методе TryMakeMove. В нём выполняются проверки корректности действия: ограничение по количеству ходов (MoveCount, MaxMoves), занятость клетки (CurrentShape != null) и принадлежность координат игровой области (с учётом Border). При успешной проверке вызывается метод MakeMove, после чего увеличивается значение переменной MoveCount и выполняется переключение текущего игрока (currentPlayer).

Метод MakeMove отвечает за непосредственное изменение состояния: размещение фигуры в ячейке и запуск пересчёта очков. Для этого вызывается метод Count, который возвращает объект Scores. Полученные значения добавляются к общим счётчикам Player1Score и Player2Score.

Метод Count реализует логику начисления очков и выполняет локальный обход окрестности вокруг позиции хода. Для каждой клетки, соседней с той клеткой, куда была только что поставлена фигура, а так же для той клетки, куда поставлена фигура, анализируется конфигурация соседей в пределах области 3×3 , подсчитываются типы фигур (BaseType) и определяется выполнение условий начисления очков..

В результате класс Board реализует полный цикл изменения состояния игры: от проверки допустимости действия (TryMakeMove) до обновления состояния и пересчёта очков (MakeMove, Count). Остальные компоненты системы используют эти методы как единый интерфейс работы с игровой логикой.

3.2 Игровая среда и формирование взаимодействия с моделью

В рамках реализованной системы игровая среда выступает связующим элементом между игровым ядром и алгоритмом обучения. Она инкапсулирует логику взаимодействия с игрой, преобразует внутреннее состояние в формат, пригодный для обработки моделью, а также обеспечивает выполнение шагов игры с вычислением награды и контролем завершения партии.

Реализация среды выполнена в виде класса GameEnvironment, который предоставляет стандартный интерфейс взаимодействия: инициализацию состояния (Reset), выполнение действий агента (AgentStep), выполнение действий оппонента (OpponentStep), а также формирование наблюдения (GetObservation) и набора допустимых действий (GetLegalActions, action_masks). Такое разделение позволяет использовать среду независимо от конкретного метода обучения и обеспечивает единый формат работы с игровым процессом.

Ключевым элементом среды является механизм преобразования состояния игрового поля в многоканальное представление фиксированной размерности, используемое в качестве входных данных модели.

3.2.1 Представление состояния и формирование наблюдения

Представление состояния игровой среды в рамках класса `GameEnvironment` реализовано в виде трёхмерного тензора наблюдений типа `double[,,,]`, формируемого методом `GetObservation()`. Пространство наблюдений имеет размерность $9 \times 9 \times 18$, где первые два измерения (9×9) соответствуют игровой области доски, а третье измерение определяет 18 признаковых каналов.

Под каналом в данном контексте понимается отдельный двумерный слой тензора наблюдений (матрица размером 9×9), содержащий значения одного конкретного признака для каждой клетки игровой области. Каждый канал отражает определённую характеристику состояния – от прямого указания на тип и принадлежность фигуры до сложных вычисляемых показателей, таких как потенциал завершения фигуры или возможность немедленной победы. Совокупность всех 18 каналов образует полное многомерное описание текущего состояния игры, подаваемое на вход нейронной сети.

Формирование наблюдения осуществляется в два этапа. Сначала метод `GetObservation()` заполняет базовую информацию о состоянии поля и локальном окружении. Затем вызывается метод `ComputeNewFeatures()`, в котором производится расчёт сложных дополнительных признаков. Оба эти метода выполняют полный проход по всему полю и анализируют для каждой клетки всех её соседей.

Такой подход может показаться неэффективным, поскольку приводит к двум полным пробегам по каждой клетке и её окрестности. Однако дополнительная вычислительная нагрузка в размере примерно 729 операций

(9×9 клеток, для каждой из которых проверяются 9 позиций) остаётся незначительной тратой ресурсов. И мне хотелось разделить логику базовых каналов с более сложными. Смешивание логики сделало бы функцию слишком тяжело воспринимаемой, что повысило бы риск ошибок в коде. Функция `ComputeNewFeatures()` и так уже довольно нагружена.

Так же отдельно стоит рассмотреть причины того, почему происходит перебор всего игрового поля, вместо локального перебора клеток вокруг только что поставленной фигуры, ведь локальный перебор снизил бы количество операций. Полный просмотр всего поля вместо локального пересчёта только вокруг последней поставленной фигуры выбран с учётом перспектив развития игры. В будущем возможно введение новых типов фигур, влияние которых может распространяться не только на соседние клетки. В таком случае подход, ориентированный исключительно на локальное обновление, оказался бы недостаточно гибким. Пришлось бы переписывать значительные участки кода. Помимо этого, перебор всего поля, как было сказано в тексте выше, является незначительной в плане вычислительных затрат операцией. Таким образом, текущее решение обеспечивает необходимую расширяемость пространства наблюдений при сохранении приемлемой вычислительной сложности.

Каналы наблюдения можно разделить на две основные группы: базовые и сложные инженерные признаки.

3.2.1.1 Базовые каналы

Базовые каналы с 0 по 8, а также каналы 14 и 17, обеспечивают фундаментальное описание конфигурации игрового поля и контекста текущей партии.

К базовым каналам относятся:

- канал 0 – индикатор пустой клетки;

- каналы 1 и 2 – наличие квадрата и треугольника первого игрока соответственно;
- каналы 3 и 4 – наличие квадрата и треугольника второго игрока соответственно;
- каналы 5, 6 и 7 – нормированная доля из восьми соседних клеток, занятых фигурами агента, фигурами оппонента и граничными элементами соответственно;
- канал 8 – индикатор текущего активного игрока;
- канал 14 – нормированное количество оставшихся ходов до окончания игры;
- канал 17 – индикатор роли обучаемого агента в текущей партии.

Базовые каналы формируют основу представления состояния игры. Они обеспечивают нейронную сеть исчерпывающей информацией о непосредственной конфигурации каждой клетки и локальной обстановке на доске. Каналы 0 – 4 позволяют модели точно определять не только наличие фигуры в клетке, но и её геометрический тип (квадрат или треугольник), а также принадлежность конкретному игроку. Такое раздельное one-hot-подобное кодирование исключает неоднозначность восприятия и даёт сети чёткое различие между четырьмя возможными вариантами занятости клетки.

Каналы 5 – 7 дополняют картину, предоставляя нормированные характеристики окружения каждой клетки. Они показывают как много фигур агента, соперника или границ вокруг определенной клетки. Данная информация особенно важна для понимания пространственной структуры игры, поскольку многие тактические решения зависят именно от контроля соседних позиций.

Канал 8 указывает на то, кто из двух игроков на данный момент ходит, что позволяет модели учитывать очередность действий сторон. Канал 14, в свою очередь, вводит глобальный временной контекст, сообщая агенту, сколько ходов осталось до принудительного окончания партии. Наличие этого признака помогает сети адаптировать стратегию в зависимости от стадии игры – вести более агрессивную игру на ранних этапах или переходить к точному позиционному противостоянию ближе к концу.

Отдельное значение имеет канал 17, который фиксирует роль обучаемого агента (за какого игрока он выступает). Этот канал критически важен при обучении через self-play, поскольку позволяет использовать одну и ту же нейронную сеть для игры как за первого, так и за второго игрока без необходимости изменения архитектуры или весов модели.

В совокупности базовые каналы создают полное и однозначное низкоуровневое описание состояния игрового поля, служащее фундаментом для дальнейшего восприятия более сложных инженерных признаков.

3.2.1.2 Сложные каналы

К сложным инженерным каналам относятся:

- канал 9 – нормированное количество квадратов среди восьми соседних клеток;
- канал 10 – нормированное количество треугольников среди восьми соседних клеток;
- канал 11 – доля заполненных клеток в окрестности рассматриваемой клетки;
- канал 12 – рассчитанный потенциал образования закрытой области квадрата в данной области;
- канал 13 – рассчитанный потенциал образования закрытой области треугольника в данной области;

– канал 15 – индикатор возможности немедленного выигрыша при постановке квадрата в рассматриваемую клетку;

– канал 16 – индикатор возможности немедленного выигрыша при постановке треугольника в рассматриваемую клетку.

Каналы 9, 10 и 11, хотя и рассчитываются в методе `ComputeNewFeatures()`, не являются сложными для расчёта каналами. В определенной мере они являются промежуточными признаками для более сложных. И используются в `ComputeNewFeatures()`, а не в `GetObservation()` именно по причине того, что используются при их расчете. Однако при этом данные каналы все равно являются важными и дают много полезной информации.

Наиболее значимыми являются каналы 12 и 13, которые представляют собой рассчитанные потенциалы образования квадрата и треугольника соответственно. Эти признаки учитывают текущее количество фигур соответствующего типа и количество пустых клеток в окрестности, формируя эвристическую оценку перспективности клетки для постановки своей фигуры. По сути, они позволяют модели получать готовую оценку тактической ценности каждой позиции без необходимости самостоятельно производить сложные комбинаторные вычисления.

Каналы 15 и 16 выступают бинарными индикаторами наличия возможности немедленной победы при постановке квадрата или треугольника в данную клетку. Их наличие позволяет сети явно выделять позиции высокой важности, где один корректный ход может привести к окончанию игры.

Таким образом, каналы 9–16 образуют единую группу высокоуровневых признаков. Каналы 9, 10 и 11 отражают локальную структуру окружения клетки и служат фундаментом для расчёта более сложных каналов. Каналы 12, 13, 15 и 16 представляют собой уже

сформированные характеристики, напрямую связанные с основными правилами игры – завершением областей и возможностью выиграть партию одним ходом.

Введение данных признаков направлено на обогащение пространства наблюдений информацией более высокого уровня абстракции. Предполагается, что предоставление модели таких заранее вычисленных характеристик должно способствовать ускорению обучения и улучшению качества принимаемых решений, особенно на ранних этапах тренировки, когда способность нейронной сети самостоятельно выявлять сложные закономерности ещё ограничена.

3.2.2 Пространство действий и их декодирование

Пространство действий в разработанной игровой среде является дискретным и имеет фиксированный размер. Количество возможных действий определяется константой `ActionSize` и равно 162. Такое значение обусловлено наличием 81 внутренней клетки игровой области (9×9) и двумя возможными типами фигур, которые может разместить агент – квадратом и треугольником.

Каждое действие представляется целым числом в диапазоне от 0 до 161 включительно. Преобразование числового значения действия в конкретный ход на игровом поле осуществляется методом `DecodeAction(int action)`.

Декодирование действия выполняется по следующим формулам:

$$pos = action \bmod 81, \quad (34)$$

$$x = Board.Border + (pos \bmod 9), \quad (35)$$

$$y = Board.Border + (pos \bmod 9), \quad (36)$$

где `pos` – промежуточная позиция в пределах 81 клетки, а `x` и `y` – координаты клетки на доске с учётом отсутствия границы (`Board.Border`).

Тип фигуры определяется диапазоном значения действия: действия от 0 до 80 соответствуют размещению квадрата, а действия от 81 до 161 – размещению треугольника.

Для работы с пространством действий в классе `GameEnvironment` реализованы следующие методы:

- `GetLegalActions()` – возвращает список всех легальных действий на текущем состоянии доски;
- `IsValidAction(int action)` – проверяет легальность отдельного действия;
- `action_masks()` – формирует булеву маску допустимых действий.

Метод `GetLegalActions()` перебирает все 162 возможных действия и включает в результат только те, для которых целевая клетка является пустой. Полученная маска действий используется при обучении модели с подкреплением для исключения недопустимых действий.

Метод `IsValidAction(int action)` использовался для отладки кода и в обучении не участвует. Сейчас данный метод нигде не используется, но оставлен, так как в будущем, при дальнейшем работе над проектом, может вновь пригодиться.

Такое представление пространства действий обеспечивает однозначность декодирования, простоту реализации и удобство интеграции с алгоритмами обучения. Одновременно оно остаётся достаточно гибким для возможного расширения в случае добавления новых типов фигур.

3.2.3 Организация шага взаимодействия

Шаг взаимодействия с игровой средой в классе `GameEnvironment` реализован через отдельную обработку действий агента и соперника с использованием методов `AgentStep(int action)` и `OpponentStep(int action)`.

Такой подход позволяет явно задать последовательность «ход агента – ход соперника» и корректно синхронизировать обновление состояния среды.

На этапе хода агента в метод `AgentStep` передаётся дискретное действие, которое декодируется с помощью `DecodeAction(int action)` в структуру `ActionInfo`, содержащую координаты клетки и тип фигуры. После этого выполняется попытка применения действия через метод `MakeMove`, что приводит к обновлению внутреннего состояния игрового поля. После чего вычисляется локальная награда с помощью функции `LocalReward()`.

Завершение хода происходит в методе `OpponentStep`, который реализует ход второго игрока. В начале выполняется проверка состояния игры через `IsGameOver()`. Если игра уже завершена после хода агента, дальнейшие действия не выполняются, и возвращается итоговое вознаграждение, вычисляемое методом `CalculateReward` (данный метод даёт награду или штраф только за окончание партии, в зависимости от результата).

В противном случае действие соперника декодируется через метод `DecodeAction` и применяется к игровому состоянию с помощью метода `TryMakeMove`. После выполнения хода производится вычисление вознаграждения методом `CalculateReward`, а также определяется факт завершения игры (`terminated`) и достижение ограничения по числу ходов (`truncated`).

Формирование действия соперника организовано с учётом различных режимов взаимодействия. Если действие заранее задано через метод `SetOpponentAction`, оно используется при выполнении шага. Такой механизм применяется при организации `self-play`, когда в роли соперника выступает другая версия обучаемой модели. В противном случае действие выбирается случайным образом из множества допустимых ходов,

формируемого методом `GetLegalActions()`. Данный режим используется на начальном этапе обучения до появления обученной модели.

Таким образом, один шаг взаимодействия с точки зрения обучения представляет собой последовательность из двух операций: выполнения действия агентом и ответного действия соперника, после чего производится вычисление вознаграждения и фиксация состояния среды. Такая организация обеспечивает корректное накопление опыта в условиях двустороннего игрового процесса.

3.2.4 Функция вознаграждения и условия завершения

Функция вознаграждения в игровой среде реализована как комбинация локальной награды `LocalReward()` и итоговой награды эпизода `CalculateReward()`. Локальная награда вычисляется на каждом ходе и отражает промежуточную оценку качества действия, тогда как итоговая награда формируется после завершения игровой партии и определяет общий результат взаимодействия агента и соперника.

Итоговая награда задаётся на уровне завершённого эпизода. Базовое значение вознаграждения равно небольшому отрицательному штрафу (-0.02), что стимулирует сокращение длительности партии. Основной вклад в итоговую награду определяется результатом игры на основе текущего счёта игроков `GetScores()`.

При достижении лимита в 80 ходов фиксируется ничейный результат без дополнительного вознаграждения. В случае победы агента добавляется положительная награда ($+1.0$). В случае поражения применяется штраф, зависящий от длительности партии: более раннее поражение штрафуются сильнее (до -3.5), тогда как при более длительной игре штраф уменьшается (до -1.0). Такой механизм повышает устойчивость стратегии и снижает склонность агента к пассивному поведению.

Дополнительно в процессе взаимодействия агента со средой используется локальная награда LocalReward(), которая вычисляется после каждого хода и отражает промежуточную оценку качества совершённого действия. Данный компонент вознаграждения направлен на формирование более плотного сигнала обучения и ускорение освоения базовых игровых закономерностей.

Локальная награда учитывает как улучшение собственной позиции агента, так и влияние хода на потенциальные возможности соперника. Положительные значения присваиваются за формирование перспективных локальных структур, усиление собственных игровых позиций и нарушение стратегически значимых комбинаций противника.

Отрицательные значения, в свою очередь, назначаются за разрушение собственных структур, создание угроз немедленного проигрыша, а также ухудшение локальной позиции в окрестности совершённого хода. Дополнительно вводятся сильные штрафы за ситуации, приближающие противника к немедленной победе.

Значение локальной награды ограничивается по модулю для предотвращения доминирования отдельных локальных эффектов над итоговой наградой эпизода и затем суммируется с основной функцией вознаграждения.

3.2.5 Итоговая организация игровой среды

Реализованная игровая среда в классе GameEnvironment представляет собой целостную систему, объединяющую представление состояния, пространство действий и механизм пошагового взаимодействия. Все ключевые компоненты среды согласованы между собой и обеспечивают корректное воспроизведение игрового процесса в рамках формализованной модели.

Состояние среды задаётся через тензор наблюдений, формируемый методами `GetObservation()` и `ComputeNewFeatures()`, что позволяет учитывать как базовые характеристики игрового поля, так и дополнительные производные признаки. Пространство действий фиксировано (`ActionSize = 162`) и связано с игровым полем через однозначное преобразование, реализованное в методе `DecodeAction`.

Механизм взаимодействия построен на последовательном выполнении методов `AgentStep` и `OpponentStep`, что обеспечивает корректную смену игроков и согласованное обновление состояния. Расчёт вознаграждения и проверка условий завершения выполняются централизованно, что исключает противоречия между отдельными этапами взаимодействия.

В результате игровая среда представляет собой замкнутую и самодостаточную подсистему, обеспечивающую корректное формирование данных для обучения и последующее использование модели без изменения базовой логики игрового процесса.

3.3 Организация процесса обучения

Обучение модели организовано на стороне Python с использованием библиотеки `Stable Baselines3` и её расширения для маскирования действий. Взаимодействие с игровой средой, реализованной на C#, осуществляется через обёртку `GameEnvWrapper`, которая реализует интерфейс `gym.Env` и обеспечивает совместимость с используемыми алгоритмами обучения. На рисунке 3 представлена архитектура процесса обучения:



Рисунок 3 – Архитектура процесса обучения

Создание экземпляра среды выполняется через механизм `pythonnet`, позволяющий загружать сборку `ConsoleGameLibrary.dll` и вызывать конструктор класса `GameEnvironment`. При этом каждая новая среда создаётся методом `_create_csharp_env()`, что гарантирует изоляцию эпизодов и корректную работу с состоянием.

Методы `reset()` и `step()` в обёртке напрямую соответствуют логике, реализованной в C#-классе. Метод `reset()` вызывает `GameEnvironment.Reset()`, после чего, в случае если агент играет вторым (`agentPlayer == 2`), инициируется первый ход соперника через `OpponentStep()`. Это позволяет привести начальное состояние к корректной игровой позиции перед началом эпизода.

Метод `step()` реализует полный цикл взаимодействия в рамках одного шага обучения. Сначала вызывается `AgentStep(action)`, после чего обязательно выполняется `OpponentStep`, независимо от того, завершилась ли игра после хода агента. Такое решение синхронизировано с логикой среды, где именно `OpponentStep()` возвращает финальные значения награды,

флагов завершения (`terminated`, `truncated`) и дополнительной информации. Итоговое наблюдение для агента формируется через `GetObservationForPlayer()`.

Метод `action_masks()` в обёртке использует соответствующую функцию среды (`GameEnvironment.action_masks()`), возвращающую допустимые действия. Эти маски передаются в модель при вызове `predict()`, что исключает выбор некорректных действий и упрощает процесс обучения.

В качестве алгоритма обучения используется MaskablePPO – модификация метода Proximal Policy Optimization, поддерживающая маскирование недопустимых действий в дискретном пространстве. Выбор данного алгоритма обусловлен тем, что в задаче присутствует большое количество возможных действий (162), часть из которых в конкретный момент времени является запрещённой, и их необходимо исключать из политики выбора.

Инициализация модели задаётся набором гиперпараметров, определяющих поведение процесса обучения. Параметр `n_steps = 2048` задаёт количество шагов среды, собираемых перед обновлением политики, что обеспечивает баланс между стабильностью градиентной оценки и частотой обновления модели. Параметр `batch_size = 512` определяет размер мини-батча при обучении на собранных траекториях и влияет на сглаженность и устойчивость градиентного спуска. Значение `n_epochs = 10` задаёт число проходов по одному и тому же набору данных, что позволяет более полно использовать собранный опыт без увеличения взаимодействий со средой.

Коэффициент дисконтирования `gamma = 0.99` определяет важность будущих наград по сравнению с текущими и выбран близким к единице, так как в рассматриваемой игре долгосрочные стратегические решения имеют критическое значение. Параметр `gae_lambda = 0.95` используется в

механизме обобщённой оценки преимуществ (GAE) и задаёт компромисс между смещением и дисперсией при оценке advantage-функции. Значение $\text{clip_range} = 0.2$ ограничивает величину обновления политики на каждом шаге оптимизации, предотвращая резкие изменения стратегии и повышая стабильность обучения. Коэффициент $\text{ent_coef} = 0.05$ вводит регуляризацию энтропии, стимулируя исследование пространства действий и предотвращая преждевременную сходимость к детерминированной стратегии.

Процесс обучения организован эпизодно и включает комбинированное взаимодействие с пулом соперников различной сложности. На ранних этапах обучения используется случайный противник, что позволяет агенту сформировать базовые стратегии и избежать неустойчивого поведения в начале оптимизации. Далее обучение происходит на смеси различных типов оппонентов, формируемых динамически в процессе работы системы.

Основу процесса составляют сохранённые версии модели, которые участвуют в регулярных оценочных турнирах с использованием Ело-рейтинга. По результатам турнира формируется основной пул соперников: в него включаются шесть моделей с наивысшим Ело, четыре модели, против которых текущая версия демонстрирует наихудший процент побед, а также дополнительные случайные модели из архива сохранённых версий для обеспечения разнообразия игрового опыта.

Пул моделей для турнира формируется похожим образом. В него попадает текущая модель, 6 лучших по Ело моделей с текущего турнира, 4 модели, против которых обучаемая модель показала наихудший процент побед в предыдущем турнире. Потом вплоть до фиксированного числа пул участников турнира заполняется старыми случайными моделями. В случае пересечения категорий (например, модель одновременно входит в топ по Ело и в список проблемных оппонентов) дублирование исключается, а освободившиеся места заполняются случайными старыми моделями.

В ходе турнира каждая модель играет с каждой другой фиксированное число партий. Её моделей приводится к фиксированному значению в начале каждого турнира. Его расчет происходит непосредственно после каждой партии. В случае если победителем турнира становится не обучаемая модель, то веса победителя турнира с определенным коэффициентом копируются в обучаемую модель.

Таким образом, процесс обучения представляет собой связку C#-реализации игровой среды и Python-инфраструктуры обучения, где вся логика игры и вычисления состояния сосредоточена в GameEnvironment, а Python-часть отвечает за оптимизацию стратегии и организацию экспериментов.

3.4 Причины отказа от метода DQN

Изначально в качестве базового алгоритма обучения был выбран Deep Q-Network (DQN), поскольку задача формально относится к дискретному пространству действий и допускает применение value-based подходов. Однако в процессе экспериментов стало очевидно, что данный метод не обеспечивает стабильного и воспроизводимого улучшения стратегии.

В ходе разработки проводилась последовательная модернизация стандартного DQN. Были внедрены Double DQN для снижения переоценки Q-значений, целевая сеть (target network) для стабилизации обновлений, а также Prioritized Experience Replay для более эффективного использования переходов с высокой обучающей значимостью. Несмотря на это, поведение модели оставалось нестабильным, а качественного роста стратегии зафиксировано не было.

Основная причина неудачи связана с характером среды. В рассматриваемой задаче действие агента оказывает не локальный, а распределённый по времени и пространству эффект: один ход может влиять на возможность завершения структуры через несколько ходов и косвенно в

других частях поля. Это приводит к тому, что функция Q-оценки становится слабо локализованной и плохо аппроксимируемой стандартной нейросетевой архитектурой, используемой в DQN.

Дополнительную сложность создаёт большое число допустимых действий (162), при этом значительная их часть в конкретной ситуации является недоступной и отфильтровывается через маскирование. Для value-based подхода это приводит к повышенной дисперсии оценок, поскольку модель вынуждена аппроксимировать Q-значения для большого количества формально существующих, но фактически нерелевантных действий.

Также выявлена проблема слабой чувствительности DQN к долгосрочным стратегическим зависимостям. В данной задаче критически важно планирование на несколько ходов вперёд, поскольку выигрыш формируется не локальным действием, а последовательностью ходов, создающих и закрывающих структуры. DQN, ориентированный на жадное максимизирование Q-значения, в таких условиях склонен к локально оптимальным, но стратегически невыгодным решениям.

В результате даже после внедрения улучшенных модификаций удалось достичь лишь ограниченного прогресса: винрейт против случайного соперника не стабилизировался выше 60%, а поведение модели оставалось нестабильным при изменении начальных условий и противников.

На основе полученных результатов был сделан вывод о том, что value-based подход не является оптимальным для данной задачи. В связи с этим было принято решение перейти к policy-gradient методу PPO, который напрямую оптимизирует стратегию действий и лучше справляется с задачами с длинным горизонтом планирования и сложной структурой вознаграждения.

3.5 Оценка качества обученной модели

Оценка качества обученной модели в рассматриваемой задаче выполняется эмпирическим способом на основе серий игровых матчей. Такой подход обусловлен отсутствием заранее известной оптимальной стратегии и невозможностью аналитически оценить качество поведения агента в сложной игровой среде. Основным критерием эффективности модели является её способность стабильно побеждать других соперников в процессе игры.

На ранних этапах обучения, помимо турнира, оценка модели проводилась через серию матчей против соперника, совершающего случайные действия. Такой подход позволял определить, формирует ли агент базовые игровые стратегии и способен ли использовать очевидные ошибки противника. По мере обучения процент побед модели против случайного соперника быстро возрастал и достигал высоких значений. После этого дальнейшее тестирование против случайного агента было признано малоинформативным, поскольку подобный соперник перестал представлять сложность для обученной модели. Результаты этого тестирования представлены на рисунке 4:

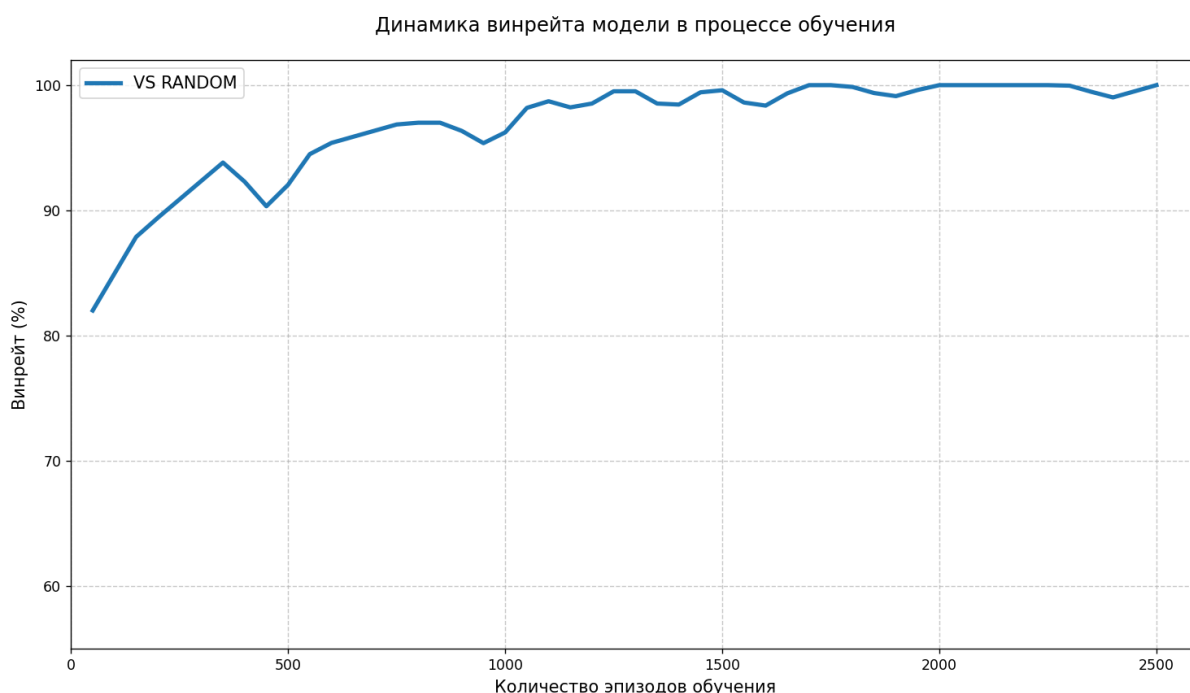


Рисунок 4 – Процент побед против соперника, делающего случайные ходы

Основной оценкой качества модели в процессе обучения являлась турнирная система с Ело-рейтингом. Через фиксированные промежутки обучения текущая версия модели сохранялась и участвовала в серии партий против других моделей из общего пула соперников. После каждой сыгранной партии выполнялся перерасчёт Ело-рейтинга участников, что позволяло оценивать относительную силу различных версий агента.

Использование Ело оказалось более информативным по сравнению с отдельными тестами против фиксированного соперника. В отличие от оценки через винрейт против одной конкретной модели, турнирная система позволяла учитывать результаты сразу против множества различных стратегий и уменьшала влияние особенностей отдельного противника на итоговую оценку качества.

Стоит отметить, что стандартные внутренние метрики обучения с подкреплением, такие как средняя награда за эпизод или средняя длина партии, в рамках данной задачи оказались слабо интерпретируемыми. Это связано с использованием динамически изменяющегося набора соперников

в процессе обучения, из-за чего распределение игровых ситуаций постоянно изменялось. В результате значения подобных метрик существенно колебались и не позволяли объективно оценивать реальное качество стратегии. По этой причине основным критерием эффективности модели был выбран процент побед в серии матчей против других версий агента.

После завершения обучения дополнительно была проведена серия матчей финальной версии модели против ранее сохранённых версий агента, полученных на разных этапах обучения. Это позволило наглядно оценить общий прогресс обучения и сравнить качество итоговой стратегии с более ранними вариантами модели. Результаты этого тестирования представлены на рисунке 5:

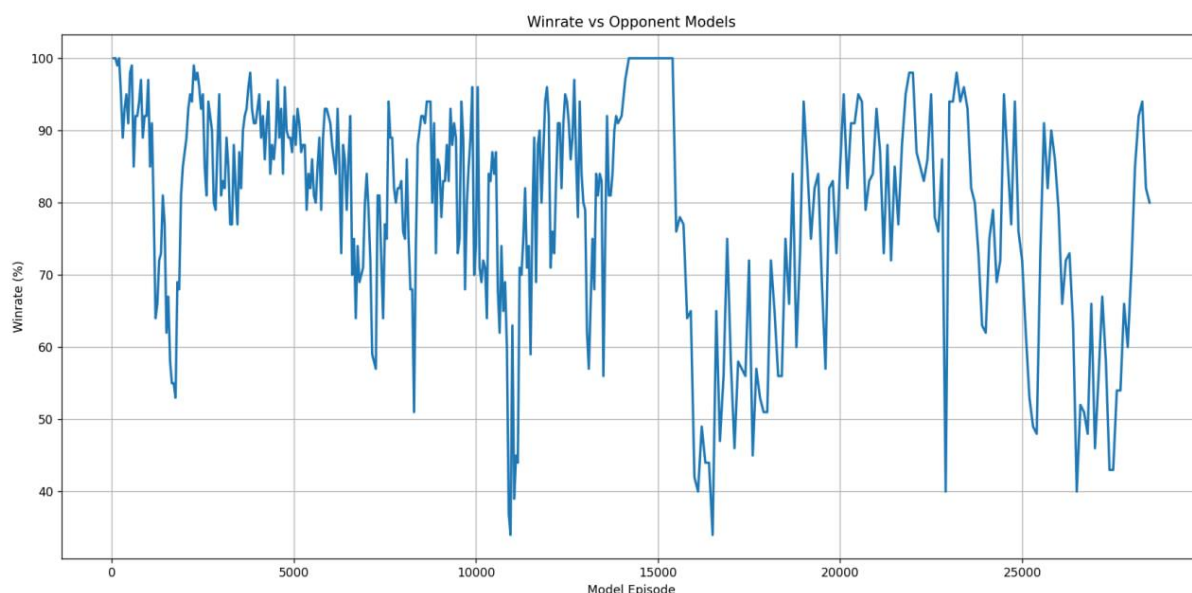


Рисунок 5 – Процент побед финальной модели против прошлых версий

На графике видно, что финальная версия модели демонстрирует высокий процент побед против большинства предыдущих версий агента. Для значительной части моделей винрейт находится выше 80 %, а на отдельных участках достигает 100 % или очень близок к этому значению, что говорит о существенном улучшении стратегии в процессе обучения.

Однако график также показывает, что обучение происходило нестабильно. В процессе self-play отдельные версии модели периодически формировали специфические игровые стратегии, против которых даже финальная модель играла заметно хуже. Именно поэтому на графике присутствуют резкие локальные провалы винрейта вплоть до значений около 40–50 %.

Подобное поведение хорошо демонстрирует одну из ключевых особенностей обучения с подкреплением: модель не движется к качественной стратегии линейно. В разные периоды обучения агент может как улучшать отдельные аспекты поведения, так и временно терять ранее сформированные игровые паттерны. Особенно заметно это в областях графика, где после высоких значений винрейта наблюдаются резкие падения качества игры против некоторых промежуточных версий модели.

При этом важно отметить, что даже несмотря на наличие локальных провалов, финальная модель сохраняет преимущество над подавляющим большинством предыдущих версий. Это говорит о том, что в ходе обучения агент постепенно формировал более устойчивую и универсальную стратегию, способную эффективно противостоять различным стилям игры, возникавшим на разных этапах self-play.

3.6 Организация пользовательского взаимодействия

Пользовательское взаимодействие с системой реализовано в среде Unity и включает три уровня: главное меню, сцену с выбором ИИ-соперника (если был выбран режим игры против ИИ) и управление игровым процессом. Главное меню предоставляет базовые сценарии работы – запуск игры с другим игроком, переход к режиму игры против ИИ и выход из приложения. При выборе режима игры с другим игроком создаётся стандартное игровое поле без использования обученной модели.

Основная логика взаимодействия в процессе игры сосредоточена в компоненте BoardUI, который выступает связующим звеном между визуальной сценой и игровым ядром (Board). В методе Start() происходит инициализация состояния: создаётся экземпляр игрового поля, после чего формируется его визуальное представление в виде двумерного массива объектов cells. Каждая клетка создаётся на основе заранее подготовленного шаблона объекта, через вызов Instantiate. При этом учитываются параметры Board.Size и Board.Border, что позволяет отображать только игровую область.

Обработка пользовательского ввода реализована в методе Update(). С использованием Mouse.current и механизма Physics.Raycast определяется объект сцены, по которому был произведён клик. Далее координаты выбранной клетки сопоставляются с логическими координатами игрового поля, после чего вызывается метод ClickCell(x, z).

Внутри ClickCell выполняется попытка хода через вызов gameBoard.TryMakeMove, который проверяет корректность действия с точки зрения правил игры. При успешном ходе на сцене создаётся визуальный объект фигуры на основе выбранного шаблона (currentPiecePrefab) с помощью Instantiate, после чего обновляется отображение счёта (UpdateScoreUI()). В случае некорректного хода действие игнорируется.

Отображение текущего состояния реализовано через текстовые элементы интерфейса (TextMeshProUGUI), обновляемые в методе UpdateScoreUI(). Дополнительно метод PrintScoreToConsole() выводит диагностическую информацию (счёт и номер хода), что используется для контроля корректности работы системы.

Выбор типа фигуры осуществляется через текущий шаблон currentPiecePrefab, который может изменяться с помощью метода

SetCurrentPiece. Преобразование визуального представления фигуры в её логический тип выполняется в методе GetShapeFromPrefab, где по имени шаблона определяется соответствующий ShapeType.

Метод DrawGridLines(), создаёт линии сетки с помощью компонента LineRenderer. Это улучшает визуальное восприятие структуры поля и упрощает взаимодействие пользователя с системой.

Таким образом, компонент BoardUI объединяет обработку ввода (Update()), применение действий (ClickCell), визуализацию объектов (Instantiate, DrawGridLines()) и синхронизацию с игровым состоянием (Board), обеспечивая корректное и наглядное взаимодействие пользователя с системой.

3.7 Применение обученной модели в игровом процессе

Применение обученной модели в игровом процессе организовано через отдельный сценарий пользовательского выбора параметров игры. Пользователю предоставляется возможность выбрать модель соперника (ONNX-файл) и сторону, за которую он будет играть. Эти параметры сохраняются в настройках и используются при инициализации игровой сцены.

После запуска игры в режиме против ИИ компонент BoardUI определяет выбранные настройки и переходит в соответствующий режим работы. В методе Start() создаётся экземпляр игровой среды, инициализированный с учётом того, за какого игрока выступает модель. Далее загружается модель через класс OnnxModelRunner, которому передаётся путь к ONNX-файлу и ссылка на игровую среду.

Непосредственное применение модели происходит в методе MakeBotMove(). Перед выполнением хода текущее состояние игрового поля передаётся в среду (_gameEnv.board = gameBoard), после чего формируется

маска допустимых действий через вызов `_gameEnv.action_masks()`. Это необходимо для исключения некорректных ходов на уровне модели.

Далее вызывается метод `Predict` класса `OnnxModelRunner`, который выполняет полный цикл инференса. Внутри него сначала формируется входной тензор и преобразуется в формат `DenseTensor<float>` размерности `[1, 9, 9, 18]`. После этого выполняется запуск модели через `_session.Run`, и из результата извлекается массив значений (`logits`), соответствующих всем возможным действиям.

Ключевым этапом является применение маски действий: для всех недопустимых ходов значения логитов принудительно занижаются (`logits[i] = -10000f`), что гарантирует их исключение из дальнейшего выбора. После этого к оставшимся значениям применяется функция `softmax`, преобразующая логиты в распределение вероятностей действий.

Далее из множества допустимых действий формируется список кандидатов, вероятность которых превышает заданный порог. Если после фильтрации подходящие действия отсутствуют, выбирается действие с максимальной вероятностью среди допустимых. Итоговый ход определяется случайным выбором одного из кандидатов, что позволяет сохранить вариативность поведения модели и избежать полностью детерминированной игры. Полученный индекс действия затем преобразуется в игровое представление через метод `_gameEnv.DecodeAction()`, возвращающий структуру `ActionInfo`.

Полученное действие передаётся обратно в `BoardUI`, где выполняется через стандартный вызов `gameBoard.TryMakeMove`. После этого создаётся визуальное представление фигуры, полностью аналогично ходу человека, и обновляется интерфейс.

Логика ИИ встроена в общий игровой цикл без отдельной ветки исполнения: после хода игрока в `ClickCell` автоматически проверяется, чей

сейчас ход, и при необходимости вызывается `MakeBotMove()`. Это обеспечивает единый механизм обработки ходов и упрощает синхронизацию состояния. Общая архитектура взаимодействия компонентов при применении обученной модели показана на рисунке 6:

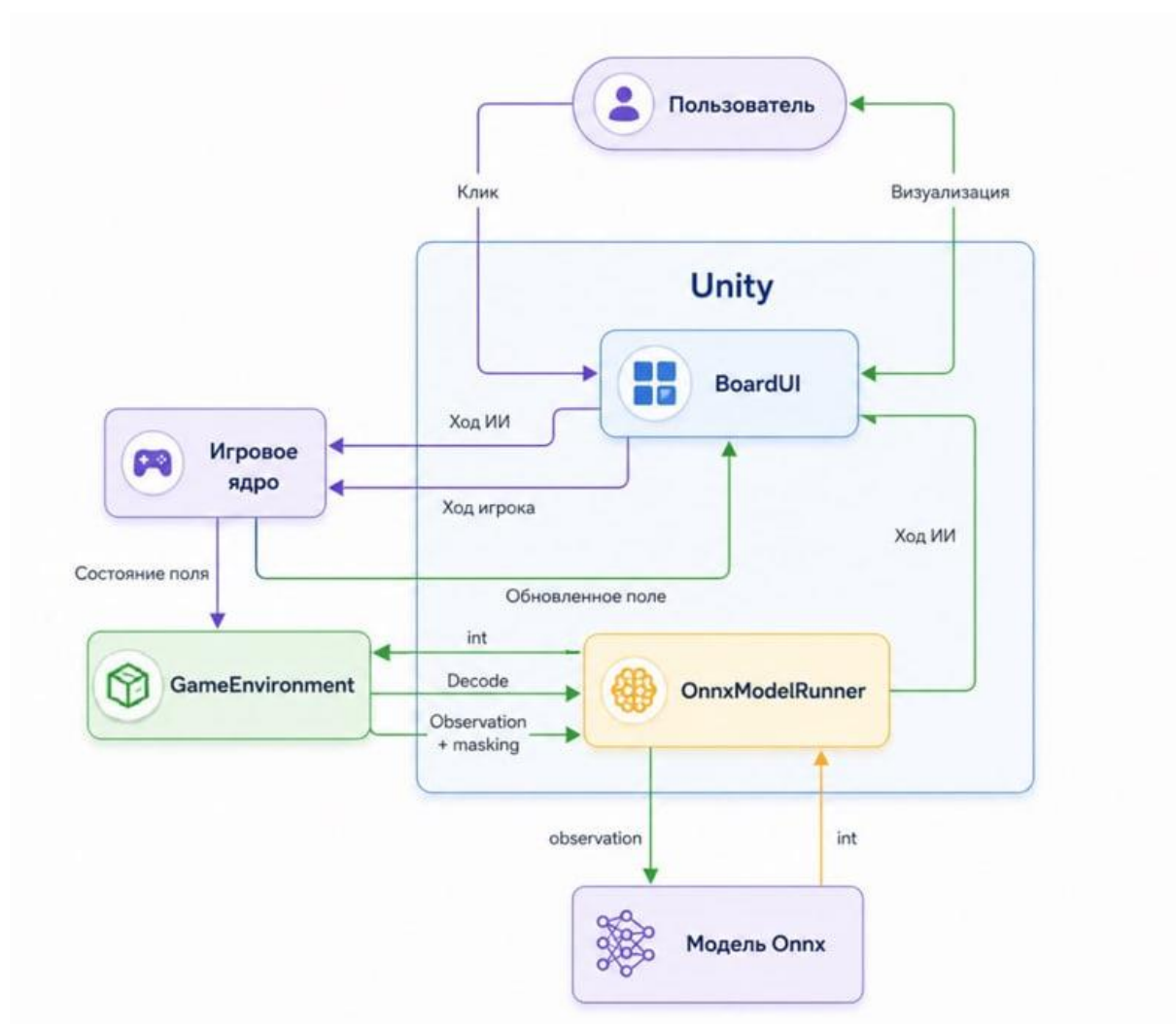


Рисунок 6 – Итоговая архитектура проекта

Таким образом, применение обученной модели реализовано как слой над существующей игровой логикой: BoardUI отвечает за интеграцию и вызов модели, OnnxModelRunner – за инференс и выбор действия, а GameEnvironment – за формирование корректного представления состояния. Такое разделение позволяет изолировать модель от интерфейса и упростить дальнейшую замену или дообучение без изменения основной логики игры.

3.8 Практическая проверка качества модели

Помимо оценки результатов обучения через турнирную систему и Elo-рейтинг, была проведена практическая проверка поведения модели в играх против человека. Необходимость этого связана с тем, что высокий процент побед означает лишь то, что модель способна побеждать свои предыдущие версии. Но это не означает, что модель сильно играет.

Проверка поведения модели против реального человека показала, что она играет слабо и допускает большое количество тактических ошибок. Проиграть модели можно, только при наличии зевков или грубых просчётов со стороны человека, поскольку в большинстве партий агент не выдерживает стабильную стратегию и делает глупые ходы даже в простых ситуациях. Так что сбор процента побед в играх против человека на данном этапе обучения является бессмысленным. Модель пока что недостаточно долго обучалась. Так что было принято решение собрать статистику о том, как модель ведет себя в различных игровых ситуациях.

Первым этапом проверки являлось тестирование способности модели находить победу в один ход. Для этого был подготовлен набор игровых позиций, в которых существовало действие, немедленно приводящее к победе. В результате было рассмотрено 50 различных случаев. Модель нашла победу в 35 из них. При этом еще в 4 случаях модель сделала победный ход через ход. Стоит отметить, что если брать в расчет игры, продлившиеся менее 10 ходов (по 5 ходов от каждого), то в них ситуация значительно лучше – из 38 таких случаев модель нашла победу в 30. И еще в 3 случаях сделала победный ход через один. Это связано с тем, что в ходе обучения длина партии в большинстве своем остается короткой и модель просто не понимает что делать после определенного количества ходов, ибо никогда раньше не бывала в подобных ситуациях.

Вторым этапом являлась проверка способности модели предотвращать поражение в один ход. В рамках данного теста использовались позиции, в которых соперник имел возможность завершить выигрышную структуру следующим действием. От модели требовалось выбрать ход, блокирующий угрозу. Здесь ситуация в целом похожа. Из 88 рассмотренных случаев модель заблокировала победу сопернику в 65. При этом стоит отметить, что иногда она блокировала выигрыш сопернику по несколько ходов подряд. Самая большая серия была длиною в 4 блокировки подряд. Ну и опять же лучше всего модель блокирует выигрыши в первые 10 ходов партии. 70 случаев было рассмотрено в первые 10 ходов и из них было 59 блокировки. При этом в 5 из этих 88 случаев модель вместо блокирования выигрыша сама закрывала область своему сопернику.

Таким образом, проведённая практическая проверка продемонстрировала, что текущая версия модели обладает ограниченной, но устойчивой способностью решать локальные тактические задачи. Агент достаточно уверенно распознаёт немедленные выигрыши и предотвращает очевидные угрозы соперника, особенно в начале партии. При этом модель проявляет заметные трудности с поддержанием стабильной стратегии и принятием оптимальных решений в сложных позициях.

Полученные результаты позволяют заключить, что основным ограничивающим фактором на текущем этапе является недостаточная продолжительность и разнообразие игрового опыта во время обучения. Модель хорошо освоила короткие эпизоды, но пока слабо обобщает накопленные знания на более сложные и длительные игровые ситуации.

Несмотря на имеющиеся ограничения, эксперименты подтверждают, что у модели сформировано корректное базовое представление об игровых закономерностях: она способна распознавать немедленные тактические

возможности, реагировать на угрозы соперника и в отдельных случаях проводить многоходовые защитные последовательности. Это свидетельствует о принципиальной правильности выбранного подхода к обучению.

В целом результаты практической проверки указывают на существенный потенциал дальнейшего улучшения модели. Одним из ключевых направлений является увеличение объёма обучения с акцентом на расширение разнообразия игровых ситуаций. Кроме того, перспективной представляется идея использовать противника, основанного на дереве решений с небольшой глубиной. Такой подход позволит модели целенаправленно отрабатывать навыки блокирования немедленных выигрышей соперника, а также построения собственных вилок и многоходовых тактических угроз. Без подобной практики победить хорошо настроенное дерево решений практически невозможно, поскольку оно стабильно реализует именно такие приёмы.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы была успешно достигнута поставленная цель разработка на основе методов обучения с подкреплением системы искусственного интеллекта для позиционной пошаговой дуэльной стратегии.

Все поставленные задачи были выполнены в полном объёме. Проведён анализ предметной области и существующих подходов к созданию игровых интеллектуальных систем. Выполнена формализация правил разработанной игровой системы, включая проверку их корректности и типизацию в рамках теории игр. Рассмотрены методы обучения с подкреплением и реализована система искусственного интеллекта на основе алгоритма Proximal Policy Optimization.

Разработана и реализована полноценная игровая среда на языке программирования C#, включающая модель состояния игры, пространство наблюдений с 18-канальным представлением, пространство действий и механизм self-play. Проведено обучение модели с использованием библиотеки Stable Baselines3 на языке Python. Выполнено тестирование обученной модели, проведена оценка её эффективности и проанализировано качество принимаемых решений в различных игровых ситуациях. Разработанное решение успешно интегрировано в программную реализацию игрового продукта.

В результате работы создана функционирующая система искусственного интеллекта, способная самостоятельно вырабатывать стратегию поведения в оригинальной позиционной игре без использования заранее заданных эвристических правил. Модель демонстрирует осмысленное поведение и способность учитывать как локальные тактические возможности, так и общую стратегическую картину игры.

Перспективы дальнейшего развития работы включают дальнейшее повышение качества модели за счёт увеличения объёма обучения и более тонкой настройки функции вознаграждения. Кроме того, планируется усложнение самой игровой механики путём добавления новых типов фигур с уникальными свойствами, расширения пространства действий и введения дополнительных правил взаимодействия между фигурами. Такие изменения позволят сделать игру более глубокой и создадут новые вызовы для системы искусственного интеллекта, способствуя развитию более сложных и обобщаемых стратегий.

Таким образом, в ходе выполнения выпускной квалификационной работы была разработана работоспособная и перспективная система искусственного интеллекта на основе методов обучения с подкреплением, подтвердившая свою эффективность в условиях сложной позиционной игровой среды.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Huth M., Ryan M. Logic in Computer Science: Modelling and Reasoning about Systems. 2nd ed. – Cambridge : Cambridge University Press, 2004. – 440 с.
2. Mnih V., Kavukcuoglu K., Silver D. и др. Human-level control through deep reinforcement learning // Nature. – 2015. – Т. 518, № 7540. – С. 529–533.
3. Osborne M. J. An Introduction to Game Theory. – New York : Oxford University Press, 2004. – 533 с.
4. Russell S., Norvig P. Artificial Intelligence: A Modern Approach. 4th ed. – Harlow : Pearson, 2020. – 1132 с.
5. Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O. Proximal Policy Optimization Algorithms. – 2017. – URL: <https://arxiv.org/abs/1707.06347> (дата обращения: 30.04.2026).
6. Schaul T., Quan J., Antonoglou I., Silver D. Prioritized Experience Replay // 4th International Conference on Learning Representations (ICLR 2016). – San Juan, 2016.
7. Sipser M. Introduction to the Theory of Computation. 3rd ed. – Boston : Cengage Learning, 2013. – 504 с.
8. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. – Cambridge, MA : MIT Press, 2018. – 552 с.
9. Van Hasselt H., Guez A., Silver D. Deep Reinforcement Learning with Double Q-learning // Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16). – Phoenix, 2016. – С. 2094–2100.
10. Watkins C. J. C. H., Dayan P. Q-learning // Machine Learning. – 1992. – Т. 8, № 3–4. – С. 279–292.

ПРИЛОЖЕНИЕ А
QR-код репозитория с исходным кодом



Рисунок А.1 – QR-код репозитория с исходным кодом